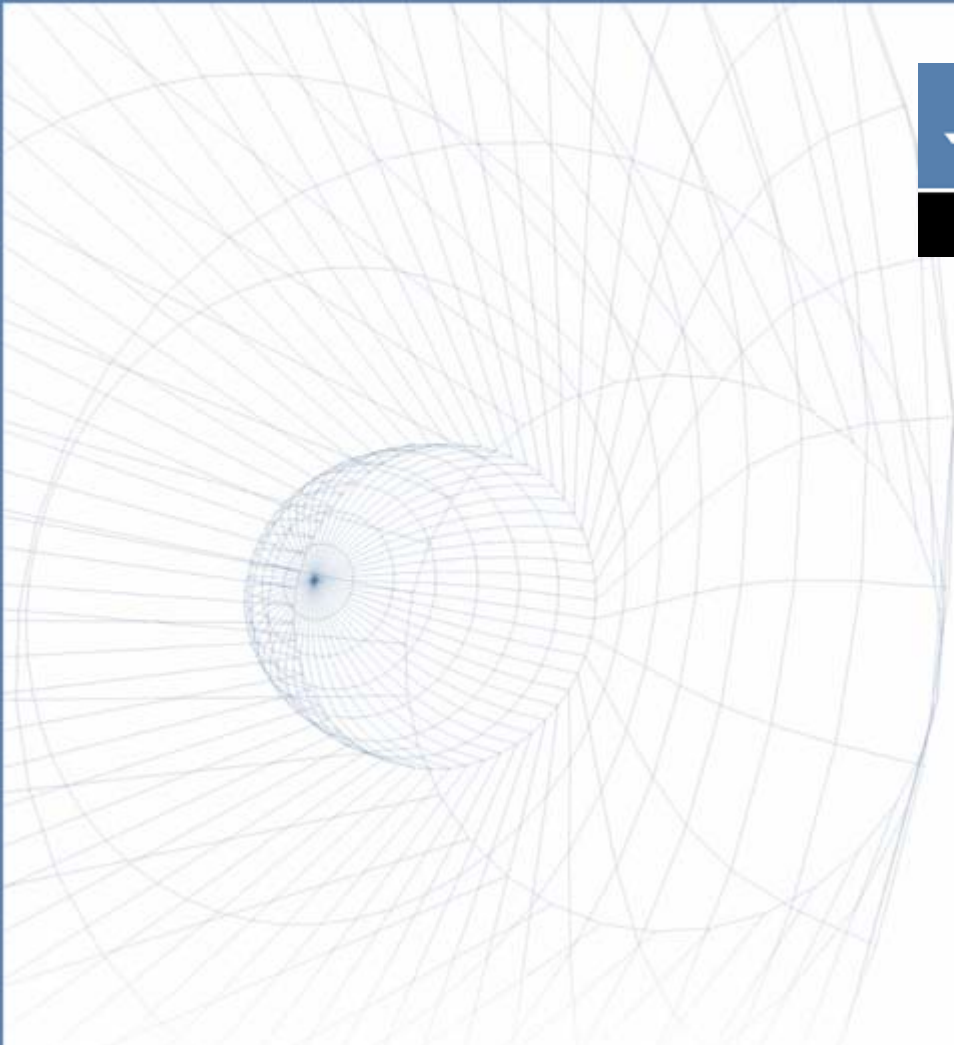




AVEVA

MARINE

12.0 to 12.1 Upgrade

Disclaimer

1.1 AVEVA does not warrant that the use of the AVEVA software will be uninterrupted, error-free or free from viruses.

1.2 AVEVA shall not be liable for: loss of profits; loss of business; depletion of goodwill and/or similar losses; loss of anticipated savings; loss of goods; loss of contract; loss of use; loss or corruption of data or information; any special, indirect, consequential or pure economic loss, costs, damages, charges or expenses which may be suffered by the user, including any loss suffered by the user resulting from the inaccuracy or invalidity of any data created by the AVEVA software, irrespective of whether such losses are suffered directly or indirectly, or arise in contract, tort (including negligence) or otherwise.

1.3 AVEVA shall have no liability in contract, tort (including negligence), or otherwise, arising in connection with the performance of the AVEVA software where the faulty performance of the AVEVA software results from a user's modification of the AVEVA software. User's rights to modify the AVEVA software are strictly limited to those set out in the Customisation Manual.

1.4 AVEVA shall not be liable for any breach or infringement of a third party's intellectual property rights where such breach results from a user's modification of the AVEVA software or associated documentation.

1.5 AVEVA's total liability in contract, tort (including negligence), or otherwise, arising in connection with the performance of the AVEVA software shall be limited to 100% of the licence fees paid in the year in which the user's claim is brought.

1.6 Clauses 1.1 to 1.5 shall apply to the fullest extent permissible at law.

1.7. In the event of any conflict between the above clauses and the analogous clauses in the software licence under which the AVEVA software was purchased, the clauses in the software licence shall take precedence.

Copyright

Copyright and all other intellectual property rights in this manual and the associated software, and every part of it (including source code, object code, any data contained in it, the manual and any other documentation supplied with it) belongs to, or is validly licensed by, AVEVA Solutions Limited or its subsidiaries.

All rights are reserved to AVEVA Solutions Limited and its subsidiaries. The information contained in this document is commercially sensitive, and shall not be copied, reproduced, stored in a retrieval system, or transmitted without the prior written permission of AVEVA Solutions Limited. Where such permission is granted, it expressly requires that this copyright notice, and the above disclaimer, is prominently displayed at the beginning of every copy that is made.

The manual and associated documentation may not be adapted, reproduced, or copied, in any material or electronic form, without the prior written permission of AVEVA Solutions Limited. Subject to the user's rights, as set out in the customisation manuals to amend PML software files contained in the PDMSUI and PDMSLIB folders and any configuration files, the user may not reverse engineer, decompile, copy, or adapt the software. Neither the whole, nor part of the software described in this publication may be incorporated into any third-party software, product, machine, or system without the prior written permission of AVEVA Solutions Limited, save as permitted by law. Any such unauthorised action is strictly prohibited, and may give rise to civil liabilities and criminal prosecution.

The AVEVA software described in this guide is to be installed and operated strictly in accordance with the terms and conditions of the respective software licences, and in accordance with the relevant User Documentation. Unauthorised or unlicensed use of the software is strictly prohibited.

© Copyright 1974 to current year. AVEVA Solutions Limited and its subsidiaries. All rights reserved. AVEVA shall not be liable for any breach or infringement of a third party's intellectual property rights where such breach results from a user's modification of the AVEVA software or associated documentation.

AVEVA Solutions Limited, High Cross, Madingley Road, Cambridge, CB3 0HB, United Kingdom.

Trademarks

AVEVA and Tribon are registered trademarks of AVEVA Solutions Limited or its subsidiaries. Unauthorised use of the AVEVA or Tribon trademarks is strictly forbidden.

AVEVA product/software names are trademarks or registered trademarks of AVEVA Solutions Limited or its subsidiaries, registered in the UK, Europe and other countries (worldwide).

3rd Party Software

The copyright, trademark rights, or other intellectual property rights in any other product or software, its name or logo belongs to its respective owner.

The following 3rd party software is included in some of the AVEVA products contained in this Online Help:

- Incorporates Teigha for .dgn files 2007-2010 by Open Design Alliance. All rights reserved.
- Microsoft® Office Fluent user interface. Fluent is a trademark of Microsoft Corporation and the Fluent user interface is licensed from Microsoft Corporation. The Microsoft Office User Interface is subject to protection under U.S. and international intellectual property laws and is used by AVEVA Solutions Limited under license from Microsoft.

[illegible]

12.0 to 12.1 Upgrade

12.0 to 12.1 Upgrade

Contents	Page
----------	------

12.0 to 12.1 Upgrade

Introduction	1:1
Part Upgrades	1:1
Upgrade Framework	1:1
Framework Functionality	1:1
Global	1:2
Upgrade Commands	1:2
The Upgrade Process	1:3
Extract Hierarchies	1:4
Working Extracts	1:5
Offline Locations	1:5
Upgrade Requirements	1:5
Part Upgrades Included in the Framework	1:5
Other Changes	1:5
Users Customisation	1:6
Part Upgrade Details	1:6
Performance of 'finding' Database Elements	1:6
Module Definitions	1:6
UKEYs (avoid duplicates)	1:6
Line Widths in Outfitting Draft	1:6
HULL FEM Data Model	1:7
Compressed HULL Objects	1:7
New Index on Hull Object Type	1:7
Start Value for Name Sequence	1:7
Assembly Drawing DB Data Model	1:7

Character Handling (Unicode Representation)	1:8
Assembly POS Attribute	1:8
Other Changes	1:8
Units in Schematic Model Manager	1:8
Shape Upgrades in Diagrams	1:8
Unicode	1:9
Creation of Quality Definitions in CATA	1:10
Hull MANU Data Model	1:11
Units (Outfitting)	1:16
Systems and CYMWRL in RefDESI DBs	1:16
Global Considerations	1:16
Units	2:1
Core Units (Outfitting)	2:2
Dimensions of Standard Stored and Derived Attributes	2:3
Dimensions and their Database Units	2:7
Units in Project Data	2:9
Units in Pre-existing Attributes of Physical Quantity	2:9
Attributes Stored as Expressions	2:13
Units in Catalogue and Design Properties	2:16
Units in Catalogue and Design Parameters	2:17
Derived Attributes	2:18
Units in Datal and Output Files	2:20
Units in Specon and Spec Files	2:20
Units and Appware	2:20
A Very Brief Introduction to Units by Example	2:20
Current Working Units and FORMAT Objects	2:22
What to look out for in PML Code	2:22
Units Qualifiers	2:24
Testing for Metric or Imperial Distance and Bore Units	2:24
Save and Restore Units	2:25
Units Conversions	2:26
Remove Units from a REAL	2:28
Units Display	2:28
Text Boxes on Forms	2:28
Dimension and Units of REAL Expressions	2:29
Other Units Considerations	2:29
Display Units	2:30
New and Modified Units PML Functions	2:31

Units in Schematic Model Manager	2:35
Dimension Support in Schematic Model Manager Prior to 12.1.	2:35
Upgrade of Dimensioned Data for Schematic Model Manager in 12.1	2:36
Units and UDAS	2:36

1 Introduction

To prepare a 12.0 project for 12.1 use, a database upgrade is required. To perform the upgrade the user must do the following:

- Start ADMIN.
- Lock the project.
- Invoke the upgrade process. Refer to [Upgrade Commands](#).
- Unlock the project.

Note: It is not a requirement that Catalogue Projects need to be upgraded. These can remain at version 12.0.

1.1 Part Upgrades

A number of changes made in 12.1 require an upgrade to parts of the data model and the database. Each individual change is referred to here as a Part Upgrade. In general these Part Upgrades have been designed to be 'optional' from a user perspective, in that the 12.1 software can work with a database that has not been upgraded and the software will degrade gracefully - that is, the software will continue to work, although new functionality may not be fully present. This means that it is possible for users to continue to work with Foreign DBs, which may be shared with 12.0 or earlier projects and which have not been upgraded, included in their projects. An example would be a Corporate Catalogue DB used for 12.0 and multiple projects.

A framework is provided to run all the part upgrades. Thus the user is provided with a single upgrade to execute - all or nothing.

- As a consequence of the 'all or nothing' approach, the project must remain in its original state if any part of the upgrade fails.

1.2 Upgrade Framework

1.2.1 Framework Functionality

The upgrade will be invoked from Admin and will control the upgrade process, and run each Part Upgrade in the appropriate order.

The upgrade process will put an upgrade number in databases, indicating the level to which they have been upgraded. This will make it easy to detect on opening whether a database has or has not been upgraded. This upgrade number will also be used by the Reconfigure.

Part upgrades outside the Framework

- Are independent of all other non-framework upgrades (i.e. non-framework upgrades can be applied in any order)

- Have a method of determining whether or not they have been applied, not relying on the upgrade number
 - This to be available to the user

It will not be possible to backtrack to pre-upgrade sessions.

1.2.2 Global

Each DB must be entirely in either an upgraded or non-upgraded state for AVEVA Marine software to operate. Therefore it is necessary that all extracts of a DB are processed during an upgrade.

The granularity of an upgrade will be a Project, excluding Foreign DBs.

1.2.3 Upgrade Commands

There is a single upgrade command which will work on a DB or the whole project. If successful, the upgrade number for the DB will be updated.

The suggested syntax is:

```
DBUP PROJECT TO LATEST
DBUP SYSTEM TO LATEST
DBUP GLOBAL TO LATEST
DBUP DB team/dbname TO LATEST
```

The user can replace LATEST with a known upgrade number which can be found using the [Q UPGRADE LIST](#) command.

```
DBUP PROJECT TO 12010101
```

Internally the code will invoke all upgrades to get to the required upgrade number. If the upgrade number is omitted, then it will be upgraded to the latest.

Any extracts will be refreshed as part of an upgrade when their Master database is upgraded.

Q UPGRADE STATUS

This command lists the current upgrade version of all databases in the project and the upgrade version that the software works on. If databases are on a lower upgrade version than the software, then the "upgrade required" text accompanies the database.

Q UPGRADE LIST

This command lists all the part upgrades ("item:" in the response) organized per upgrade version. I.e. a part upgrade belongs to a particular upgrade version. An upgrade version is the increment we do database upgrades in. The upgrade version is a 8-digit number. So to upgrade a database to a specific upgrade version, the user can give the command

```
DBUP DB MYTEAM/MYDB TO 12010103
```

This command will upgrade the database MYTEAM/MYDB to upgrade version 12010103 including the versions 12010100, 12010101, 12010102. I.e. upgrades versions are applied sequentially, and it is not possible to skip any intermediate versions.

Global Projects

In Global projects, databases must be upgraded at their primary location. The upgrade must be run separately at each project location, since any secondary databases will be ignored.

All descendant extracts must be primary at the same location as their master database, otherwise the database hierarchy will not be upgraded. Such databases can be identified using the ISEXCP attribute. If a database is primary (ISPRIM TRUE), but not all its extracts are primary (ISEXCP FALSE), then it will be omitted from a project upgrade.

Additional syntax is available in Global projects to allow for centrally administered system databases. These cannot be upgraded at the administered location, but must be upgraded at their primary location:

DBUP SYSTEM FOR locnam TO LATEST

DBUP ALLSYSTEM TO LATEST

Where locnam defines the LOCID, name or reference (gid) of a Location element in a Global project. This syntax will be available in ADMIN.

The ALLSYSTEM option in a Global project allows all primary system databases to be upgraded.

Individual satellite system databases may be upgraded using the 'SYSTEM FOR locnam' syntax provided they are primary. If the Global daemon is running, the upgrade will issue Global commands to send such administered system databases back to the administered locations.

It is the responsibility of the System administrator to make sure that updates are run to send all modified databases to satellites; and to relocate extract databases as required back to their original primary locations.

In a Global project, the UPGRADE STATUS query (see below) will also show the status of secondary databases and extract hierarchies. This will help administrators to identify which extracts will need relocating.

Note: Extract hierarchies which contain secondary extracts cannot be upgraded.

1.2.4 The Upgrade Process

The upgrade process will be undertaken by System Administrators responsible for the project at all locations. It is feasible that system administration may be taken at a remote location for some locations. When upgrading multiple projects then many System Administrators will need to co-ordinate. The upgrade process may become complicated if running through different time zones. The upgrade process will upgrade one project at a time. Consideration to the order of projects to be upgraded will need to be undertaken by the user.

The projects will need to be locked for the duration of the upgrade, with all Users out of the system.

The following upgrade steps must be performed by an administrator:

1. Make sure all users have exited from project
2. Lock project at all locations (upgrade will check for this. (see below)
3. Disable Automatic update events
4. Expunge all users in the system at the local location
5. Flush data from Working extracts - these will not be considered

6. Check for No Transient Databases
7. DICE project
8. If DICE reveals issues, address them, then re-run DICE
Administrator may want to unlock project while DICE issues are being addressed, but will need to exclude all users and Lock project again before final DICE]
9. [After clean DICE]
10. Back-up project at all locations

The following upgrade steps will performed by the upgrade:

1. At each location, run Update
2. Deep Refresh with Propagation on all DBs
3. Temporarily relocate all non-Foreign DBs, to make appear Primary at Hub
4. Loop over all non-Foreign DBs in project at Hub and Upgrade (i.e. run each framework part-upgrade on that DB)
5. Do NOT perform Savework during this process
6. Update at all locations
7. Refresh
8. Post-process at all locations
9. **Optionally** Merge Sessions
10. **Optionally** Reconfigure for Unicode
11. Update at all locations
12. Relocate DBs back to original locations
13. DICE check project
14. Perform non-framework upgrades if applicable

The mechanism by which the Administrator will tell the upgrade whether to Merge Sessions, and whether to Reconfigure for Unicode, are design details which will be described in the design documentation.

Locking the Project

The project as a whole cannot be locked, only individual locations; however, it is possible to lock all online locations from the HUB through Global. To do this run the following command from the HUB:

```
LOCK AT <location>
```

The HUB can be locked without the need for a daemon command by just typing:-

```
LOCK
```

It is possible to confirm whether locations are locked by evaluating the return result from:-

```
QUERY LOCK AT <location>
```

1.2.5 Extract Hierarchies

It should not be necessary to change the extract hierarchy, or to consolidate data within extract hierarchies. Therefore the System Administrator should not need to FLUSH, ISSUE, DROP data between extracts (working extracts are an exception to this - see below). Nor should they need to delete any extract families to leave only Masters. However all extracts will need to be relocated to a single location, although this does not need to be the HUB.

1.2.6 Working Extracts

The upgrade process will need to make sure that all data is up to date at the HUB where pre-scan data checks will need to be made. Working Extracts cannot be propagated as they are specific to a single location. As a result all data MUST be flushed, and claims released from the Working Extract into its parent. This is only true for working extracts, all other extracts do not need to be flushed, or have its claims released, as all non working extracts will be available at the HUB.

1.2.7 Offline Locations

Global supports Offline locations; therefore we cannot assume that the Hub has a Global connection to that location. Where as Offline locations do not support distributed Extracts it can support stand-alone extract families.

It will not be possible to co-ordinate the upgrade from another location if Offline locations are used. Offline locations are relatively independent, and can be treated as such.

1.3 Upgrade Requirements

The necessary database upgrades for use of 12.1 will be implemented as part upgrades. Other internal changes may be handled differently.

1.3.1 Part Upgrades Included in the Framework

The following requirements for a Part Upgrade are included in the 12.1 upgrade framework:

UKEYs (avoid duplicates)

Performance of 'finding' Database Elements

Module Definitions

Character Handling (Unicode Representation)

HULL FEM Data Model

Compressed HULL Objects

New Index on Hull Object Type

Start Value for Name Sequence

Assembly POS Attribute

Line Widths in Outfitting Draft

1.3.2 Other Changes

The following requirements for other changes are related to moving from 12.0 to 12.1

Units in Schematic Model Manager

Move CYMWRL to new REFDESI db

Shape Upgrades in Diagrams

Unicode

Hull MANU Data Model

Units (Outfitting)

1.3.3 Users Customisation

Users will have to review all their customisation, to check that assumptions are not invalidated by 12.1 changes. For example:-

- If Engineering applications require write access to existing SYSTEMs they will need to be moved to a RefDESI DB
- PML may need to be edited because of the new Outfitting Unit handling.

1.4 Part Upgrade Details

1.4.1 Performance of 'finding' Database Elements

A change has been made to significantly improve performance of finding database elements when Type is one of the criteria in the selection. This requires an Index on Type. Invisible attributes are added to all elements of relevant element types by the upgrade script and an Index is added by the upgrade script. This needs to be performed on the entire extract hierarchy.

1.4.2 Module Definitions

At 12.1 there are some changes to Module Definitions in the AVEVA Marine product.

- A new module, Tags, has been added. This is part of the Engineering Product, but will be added for all projects to enable them to adopt use of the Engineering product if and when they decide to do so.
- The module, Hull Drafting has been renamed Marine Drafting in recognition of increased use for Outfitting Drafting as well as Hull.

These module changes are made by the upgrade script

1.4.3 UKEYs (avoid duplicates)

At 12.1 the 'Database Number' is part of the UKEY when it is created. This prevents UKEY clash of any UKEYs created at 12.1.

AVEVA Marine 12.1 continues to be able to use UKEY references in 12.0 format (created at 12.0 or earlier), even when the UKEY definition has been upgraded. Thus users can 'mix' 12.0 format UKEYs and those created at 12.1. Therefore only 'UKEY' clash will be both 12.0 format. It is possible to convert UKEYs in 12.0 format to 12.1 format.

Need to change DICT DB (UKEY definition) first, then all DBs referencing it. The DICT DB change is performed by the upgrade. AVEVA recommend performing the change on UKEY references on an 'as need' basis (i.e. if a UKEY clash is encountered). This is because of the time which could be required to update all UKEY references in a project.

1.4.4 Line Widths in Outfitting Draft

A 12.0.SP6 fix addresses a Line width problem in Outfitting Draft, where 'thin', 'medium' and 'thick' lines were not the exact width expected, leading to lines which were expected to have different thicknesses having the same width. A macro was provided with fix to add necessary attributes. This is incorporated in 12.0 to 12.1 upgrade. It will do nothing where 12.0 fix macro has already been applied and will add necessary attribute in other cases

1.4.5 HULL FEM Data Model

It is envisaged that FEM Data will be stored in a new dedicated database under a FEMWLD. It is expected that each location will maintain its own FEM data in a database that is allocated at that location. The upgrade will delete all obsolete FEM data (apart from settings objects), and this will be regenerated from scratch. In terms of the upgrade, the only action taken is to delete all obsolete FEM data from existing databases, if 12.1 users wish to make use of FEM data they will create a new FEM database with a FEMWLD to contain all data.

The upgrade process will need to repopulate databases to any secondary locations where changes have been made. However, the thought is that it would be unlikely to populate FEM data to other locations.

1.4.6 Compressed HULL Objects

The part upgrade will rewrite all OBJHD hierarchies in a condensed format, which in turn will improve performance, by changing the descendants down to 10% of the original number. The redefined hierarchies will be written to the new session as created at the end of updating the db by the framework. The upgrade is expected to be transparent to the user.

1.4.7 New Index on Hull Object Type

This part upgrade will rewrite the pseudo attribute OCONE of element OBJHD so it is stored in the new integer table OCONET. The performance will be improved by increasing the speed of searching on the object code (OC1 and OC2). The upgrade is expected to be transparent to the user.

1.4.8 Start Value for Name Sequence

A new attribute SQSTRT has been added to the NAMSEQ to determine the start value of the name sequence.

The change will need to be propagated to all secondary locations where the NAMSEQ db is allocated. NAMSEQ will not have extracts as these are UPDATE databases.

1.4.9 Assembly Drawing DB Data Model

A development to supply the User with the ability to view an object from a different viewpoint and a different angle rather than relying on the world co-ordinates.

Deployment requires the use of a new ADP library which is supplied within the MAR model on the DVD. Refer to the AVEVA Hull & Outfitting 12.0.SP5.2 release letter for further information.

An upgrade script provided allows this to be updated.

The scenarios are as follows:-

- Existing 12.0.SP6 users have already upgraded, and will not need to do anything
- New Users:-
 - Only 12.0 users who have not run the upgrade in 12.0SP5.2 or later need to upgrade.

Therefore for Global projects the part upgrade will need to run the script at each location.

1.4.10 Character Handling (Unicode Representation)

See [Unicode](#).

This is an optional part of the 12.0 to 12.1 upgrade. Users can continue to operate in a 12.0 character set at 12.1, providing Language environment variables are set appropriately; but 12.0 limitations on characters allowed will then remain. This upgrade is a Reconfigure. It is an optional step in upgrade. Alternatively it can be undertaken on a separate occasion.

- AVEVA Marine 12.1 can open and read databases created prior to 12.1
 - The project setting must be correct
 - So only 1 character set in a project
- AVEVA Marine 12.1 can write to databases created prior to 12.1
 - The project setting must be correct
 - The character must be in the character set
 - An attempt to write an invalid character will result in an error

1.4.11 Assembly POS Attribute

A POS attribute is now required for ASMBLY elements and the upgrade makes sure such an attribute is added to all assemblies. Currently there is no functionality utilising positioning of assemblies, but it is required to facilitate for some general (Outfitting Draft) functionality when operating on assemblies.

1.5 Other Changes

1.5.1 Units in Schematic Model Manager

Schematic data imported into Schematic Model Manager prior to 12.1 must be upgraded to use new Units functionality, but this process will be handled separately to the main upgrade process. A check is performed automatically on entry to Schematic Model Manager and the user will be warned if an upgrade is required. The upgrade process must be carefully considered by project administrators as it can affect multiple projects and locations. Firstly, schematic data is scanned to identify changes required. Secondly, UDA definitions are updated for the appropriate units. Thirdly, the changes identified are applied to the schematic data.

Refer to Schematic Model Manager User Guide for details of the upgrade process.

1.5.2 Shape Upgrades in Diagrams

The shape upgrades for Diagrams are changes to Visio shapes. These changes are to Visio files and not the Dabacon database. They will be actioned by the Diagrams Application when the diagram is opened in write mode by the Diagrams 12.1 application.

Users will be able to choose to:

- Execute a batch job function available from within Diagrams
- Set an 'automatic upgrade' flag, so that each Diagram is upgraded when it is first opened in 12.1
- Manually call the upgrade option from the Tools menu when a non-upgraded Diagram is open. If the setting says that no automatic upgrade should be performed on open, then a warning will appear in message log, saying that the diagram needs updating.

Non-upgraded Visio shapes will still work in Diagrams 12.1, although they will not have any extended functionality, such as new context menu options etc. So Foreign 12.0 DBs can be used at 12.1

In Global/Extract scenarios the upgrade will work as any other change; the diagram will be saved in a new version after upgrade. If the upgrade is performed on an extract, it will be updated on the Main DB after flushing the extract.

1.5.3 Unicode

At 12.1 new Dabacon databases will, by default, store text in a Unicode encoding; these may be termed **Unicode encoded Databases**.

Databases created prior to Unicode enabled AVEVA Marine 12.1 to store names, text attributes and other text strings using an encoding determined by the project settings, which determines the range of characters that may be present. These may be termed **Locally encoded** or **Legacy** databases since the project settings are set to match a specific locale (Russian, Chinese etc). By default, the encoding is Ascii ISO8859-1 ("Latin 1").

Such locally encoded databases do not need to be modified or upgraded to be used in 12.1. They may be opened and read from (for example as Foreign Databases) without restriction, since the Unicode standard encompasses all existing local encodings. They may also be written to, with the restriction that character data may only contain characters in the **project**-defined encoding. An attempt to write an invalid character (for example a name containing a Chinese character into a Russian database) will be rejected with an error.

It is important that any project containing locally encoded databases (either directly or as foreign dbs) has its project settings set explicitly and correctly to make sure that character data is interpreted correctly.

Unicode encoded databases cannot be opened (for reading or writing) with pre-Unicode versions of Aveva Marine. However, it is possible to specifically create locally encoded databases if it is required that they should be accessible by previous versions of Aveva Marine.

In cases where it is required to extend the range of characters that may be used in existing locally encoded databases, RECONFIGURE may be used to convert it to a Unicode encoded database.

In the following example legacy DICT dbs (used to hold UDA and UDET names) are reconfigured to be Unicode encoded. Using a Unicode Executable (12.1) for db MASTER/DICT (In ADMIN):

```
FROM DB MASTER/DICT
TO FILE /c:\DICT1 /c:\DICT2
RCFCOPY ALL
RECONFIG SESSIONS
FROM FILE /c:\DICT1 /c:\DICT2
TO DB MASTER/DICT
RECONFIG
```

Doing it this way means that no deletion and recreation (or copy) is required for the DB, and therefore no re-adding to the MDB structures is required either. Using RECONFIG SESSIONS in the FROM phase of the reconfigure operation will preserve both the sessions and references.

In Summary:

Locally Encoded (Legacy) Databases:

- can be opened for read access in both Unicode and non-Unicode versions of Aveva Marine
- can be opened for write access in both Unicode and non-Unicode versions of Aveva Marine, but the range of characters which may be used is restricted to the set defined by the project settings
- require that the project settings are correct so that characters can be interpreted correctly
- can be reconfigured to a Unicode encoded database

Unicode Encoded Databases:

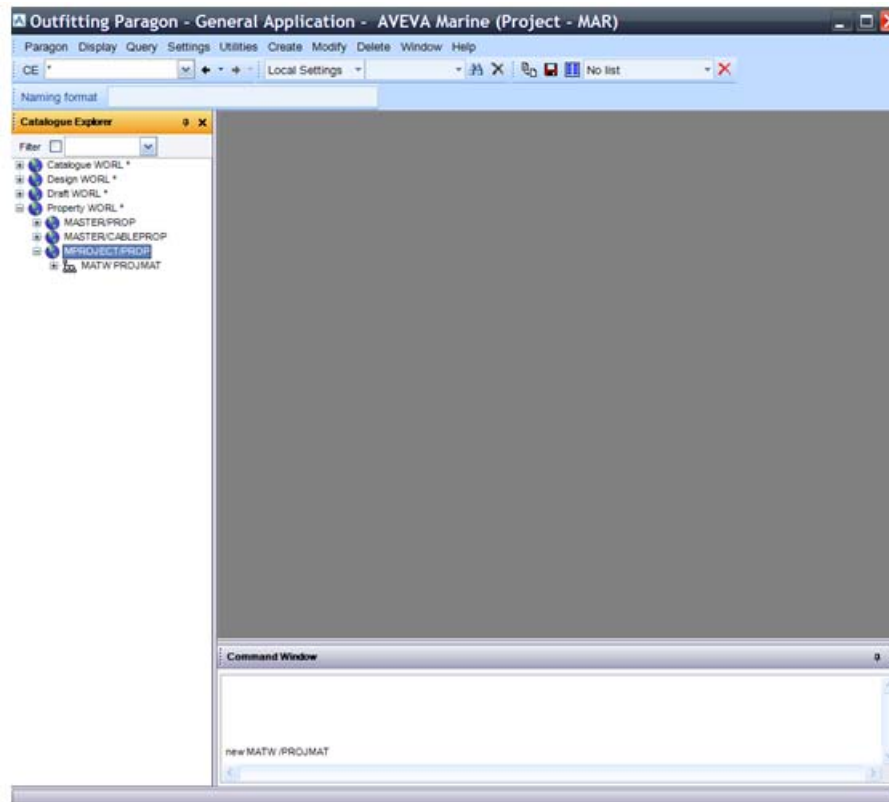
- cannot be opened for read or write access in pre-Unicode versions of Aveva Marine
- can store the full range of Unicode characters available in Unicode versions of Aveva Marine

Unicode in Marine

Marine Outfitting and Marine Drafting code will handle Unicode strings. Administrators may have chosen to convert all DBs which **do not contain Hull data** to Unicode as part of their upgrade process, or may decide for each DB whether and when to upgrade manually, and perform this upgrade using Reconfigure as in the example above.

1.5.4 Creation of Quality Definitions in CATA

A new element of type MATW must be created in the Property world. The PROP database might be read-only. It can be changed to Read-Write for Paragon in Admin. **Select Project > Module Definitions** and select Paragon from the application list. The database mode can be changed by clicking Advanced Settings. The database should be changed back to Read Only after the quality elements have been added.



The definition of the qualities, the file assigned to the environment variable SBH_QUALITY_LIST in AVEVA Marine 12.0, will now be stored in the Catalogue. A new pml-function hullCreateQualityList.pmlfnc has been developed to read the file and create the quality elements in Dabacon. It should be executed from the command line in Paragon by giving the command !!HullCreateQualityList().

The user will be prompted to select the input file from an open file dialogue. It might be necessary to change the file in the following way:

- No blanks should appear in any parameter. If this is necessary the file must be changed to a comma separated file.

In the quality elements, the Aveva Marine element SDEN is used for storing the density. In this element, the attributes for temperature and pressure are not used in the Marine applications.

1.5.5 Hull MANU Data Model

The Hull MANU data model has been completely revised for 12.1. The change of format of MANU data is mandatory; 12.1 will not operate with 12.0 format data (and 12.0 will not work with 12.1 format data). The changes include possible division of MANU data from one 12.0 MANU DB among multiple 12.1 MANU DBs. User intervention is required to coordinate how the data is divided, which effectively prevents the MANU update from being included in the upgrade framework.

When the 12.1 ManuConfig AddIn is started it will check the format of data in the MANU DBs, and if they are 12.0 format will prompt the user to update. The user can either accept or abort the upgrade of manufacturing data. If the DB is a foreign DB the user will have no alternative but to abort upgrade of manufacturing data. Because MANU DBs contain

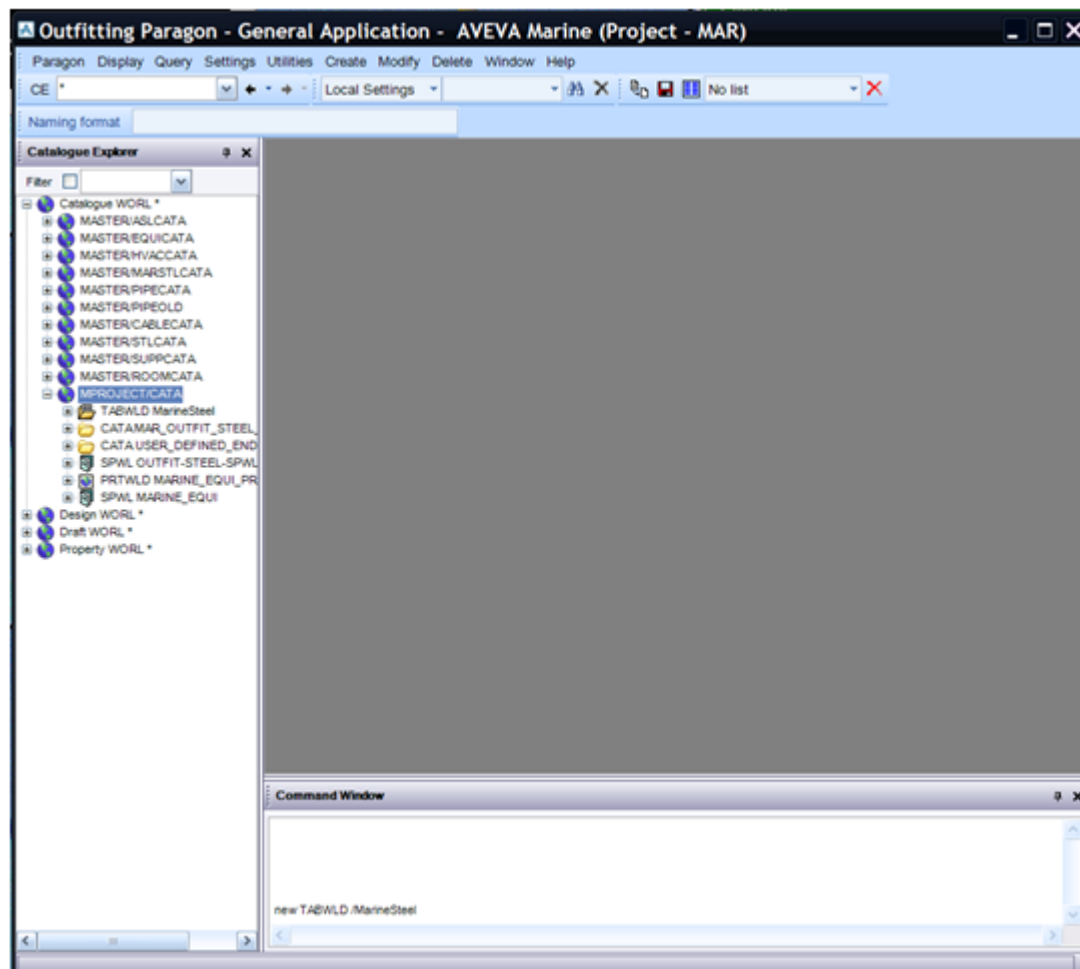
project-specific information it is not thought that they are ever shared between projects, so it is believed this restriction is acceptable for MANU DBs.

AVEVA will recommend the update of MANU DBs for the project is undertaken by the Administrator immediately after execution of the upgrade Framework. To encourage this, the upgrade framework will output a prompt, if there are MANU DBs in the project, suggesting the Administrator does this when the framework upgrade is complete.

The restructure of the Hull Data Model will have a drastic change to the existing Hull model. It appears that the upgrade will have to take on several steps.

Creation of Raw Plates Definitions in CATA

This is a very similar process to the Creation of Quality Definitions in CATA, in the previous section. A new TABWLD will be created in the Catalogue world to store the Raw Plates. The definitions will be read into the CATA database by running the function !!HullDefineRawPlates(), which will prompt for the definition file. This also needs to run in Paragon with write access to CATA databases.



Creation of MOG World in MANU Databases

There has to be a MOG world defined (MOGWLD) in the manufacturing database to be able to upgrade a 12.0 project to 12.1.SP1. This world element will hold some settings data that will be move under the upgrade from design databases to manufacturing databases.

This upgrade will need to be made in 3 steps currently this is available as a ManuConfig add-in. If upgrade is required an Upgrade Project Tab will be displayed in the UI. This tab will have the 3 steps for upgrade listed:-

- Create File
- Analyse
- Upgrade

Once the project has been upgraded the tab will disappear.

Step 1 - Create File

This function reads a template of the manufacturing package structure and creates a file to be used for the creation of all manufacturing packages in the project. One manufacturing package will be created for each MBLOCK and in the same MANU data base as that block. The template file manuPkgtemplate.mac shown below is included in the delivery (in the <PROJECT>mac directory):

INPUT BEGIN

```
NEW MANPKG /<BLOCK_NAME> DB <DB_NAME>
```

```
NEW MPKGFT /<BLOCK_NAME>Filter01
```

```
MPKGFI (MATCHWILD ( ATTRIB NAMN , '<BLOCK_NAME>*' ))
```

```
END
```

```
NEW MPKGFL /<BLOCK_NAME>PlateParts
```

```
MPKGFI ( ATTRIB TYPE EQ 'MPLATE' )
```

```
NEW MPKGFL /<BLOCK_NAME>PlanarPlates
```

```
MPKGFI ( ATTRIB TYPECD EQ 91 )
```

```
END
```

```
NEW MPKGFL /<BLOCK_NAME>DevelopedPlates
```

```
MPKGFI ( ATTRIB TYPECD EQ 92 )
```

```
END
```

```
NEW MPKGFL /<BLOCK_NAME>BracketsPlates
```

```
MPKGFI ( ATTRIB TYPECD EQ 94 )
```

```
END
```

```
NEW MPKGFL /<BLOCK_NAME>DoublingPlates
```

```
MPKGFI ( ATTRIB TYPECD EQ 81 )
```

```
END
```

```
NEW MPKGFL /<BLOCK_NAME>ConvertedProfiles
```

```
MPKGFI ( ATTRIB TYPECD EQ 87 )
```

```
END
```

```

NEW MPKGFL /<BLOCK_NAME>CollarPlates
    MPKGFI ( ATTRIB TYPECD EQ 88 )
END
NEW MPKGFL /<BLOCK_NAME>BendingTemplates
    MPKGFI ( ATTRIB TYPECD EQ 96 )
END
END

NEW MPKGFL /<BLOCK_NAME>Profiles
    MPKGFI ( ATTRIB TYPE EQ 'MPROF' )
END

NEW MPKGFL /<BLOCK_NAME>BuiltProfiles
    MPKGFI ( ATTRIB TYPE EQ 'MBPRO' )
END

NEW MPKGFL /<BLOCK_NAME>PinJigs
    MPKGFI ( ATTRIB TYPE EQ 'CPINJG' )
END

NEW MPKGFL /<BLOCK_NAME>Other
    MPKGFI ( true )
END

OLD /<BLOCK_NAME>
    MPKGRF MPKGFL /<BLOCK_NAME>Other
INPUT END /<BLOCK_NAME>

```

This macro will also create the rules for Plate Nestings and Profile Nestings.

Before creating the manufacturing package structure, the user can edit the template and change the structure. When the macro is executed in the command line, one manufacturing package will be created for each MBLOCK and <BLOCK_NAME> will be replaced by the actual block name and <DB_NAME> will be replaced by the name of the MANU data base in which the MBLOCK is stored.

The rules for Plate Nesting's and Profile nesting's will be added by the system when manPkg.mac is created. These rules can be edited before the manufacturing package structure is created.

Run the macro file from the command line (drop the file in the window) to create the manufacturing package structure.

When manufacturing packages have been created a check can be done to verify that the generated production parts will be stored in correct manufacturing package. This utility will loop over all panels in the design database 1and list in which manufacturing package the production parts will be stored.

The verify function also check plate and profiles nesting's and the result is outputted in a file placed the list directory of AVEVA marine. It should be executed from the command line in Hull Design by giving the command `!!hullCheckManpkg()`.

Step 2 - Analyse

When the manufacturing packages have been created it is possible to start the upgrading of the project. The project analysis is started from the Upgrade Project tab in the ManuConfig add-in.

In this phase, the system sequences the manufacturing databases and checks that a manufacturing package exists for each of following production parts:

- MOGWLD is defined in a manufacturing database. An error is given if the world element is missing.
- Quality list are defined in catalogue. An error is given if no quality definitions are found.
- Raw plates (elements MRAWQU, MRESTQ). A warning is given if a raw plate does not exist in the catalogue.
- Production parts (elements MPLATE, MROF, MBPRO in MBLOCK). An error is given if no manufacturing package has been found. A warning is given if the production part has no owner in the DESI data base.
- Nested plates (element MNSTQU). An error is given if no manufacturing package has been found.
- Nested profiles (element MPRNST). An error is given if no manufacturing package has been found.
- Pin Jigs (element CPINJG). These objects will be moved from the DESI data base to a suitable manufacturing package. An error is given if no owning curved panel is found.

If some elements without owner in the DESI data base are found, the user gets the possibility to remove these elements.

The result of the analysis is stored in the file `AnalyseErrorList_YYYYMMDDHHMMSS.txt` in the `SB_SHIPPRINT` directory of the project. If errors are found they must be corrected before the Project Upgrade can be started. After the corrections the analysis has to be made again.

In large projects the analysis can take some time. It is therefore possible to assign the environment variable `SBH_MANU_UPGRADE_BLOCK` to the manufacturing blocks to be treated. All other blocks will then be skipped.

The syntax is:

```
SBH_MANU_UPGRADE_BLOCK   Block:<block name>
```

To analyse all manufacturing blocks beginning with A, the following setting should be made:

```
SBH_MANU_UPGRADE _BLOCK   Block:A*.
```

Step 3 - Upgrade

When the analysis part upgrade completes with no errors it is possible to undertake the upgrade. The upgrade moves all production parts in the blocks to the new manufacturing package structure. Parts that have been moved will be reported in the `ProjectUpgradeStatusList_YYYYMMDDHHMMSS.log` in the `SB_SHIPPRINT` directory.

Upgrade of nested plates and profiles are not made until all production plates and profiles have been upgraded. As a result of the upgrade all element types of MBLOCK and MPANEL should have been deleted. If errors have occurred then the process will roll back to the last SAVEWORK state.

The upgrading of nested plates and nested profiles are not made until all production plates and profiles successfully have been upgraded.

After upgrading the project it can be necessary to merge the sessions to save disk space. This is especially important if SBH_MANU_UPGRADE_BLOCK has been used.

Limiting scope

It is anticipated that for large projects this may take a long time. It is therefore possible to undertake the upgrade in steps by using the SBH_MANU_UPGRADE_BLOCK environment variable to determine the Blocks to upgrade. This applies to the Analyse and Upgrade steps. However, all Blocks will need to be upgraded before the project can be used.

This upgrade is independent of the Upgrade process, and will be undertaken by the administrator as a separate process.

1.5.6 Units (Outfitting)

At 12.0 and earlier versions the only physical quantity which was formally recognised in Outfitting was length, used for DISTance and BORE, and the derived %SQDI (squared length) and %CUDI (cubic length) set via the UNIT field of an Attribute.

Most other Dabacon products had similar restrictions, **except** for:

- Schematic data imported via Schematic Model Manager (refer to [Units in Schematic Model Manager](#))
- Hull and Marine Drafting use the same unit handling as Tribon M3. **This will continue at 12.1, therefore no upgrade or unit conversion is required.**

1.5.7 Systems and CYMWRL in RefDESI DBs

Neither Systems nor CYMWRLs will be put in RefDESI DBs by the 12.1 upgrade script, although AVEVA would encourage Administrators to move them to RefDESI DBs to enable users to make maximum advantage of new features in 12.1.

Systems can still be placed in DESI DBs at 12.1 - and users without any of the Engineering or Diagrams products may choose to do this. Where Systems are in DESI DBs, Diagrams and Engineering products can still assign elements to them. If Users want to move Systems to a RefDESI DB they should be able to do this with normal copy/move commands. Any problems encountered doing this should be regarded as Defect Fixing. Therefore it was not necessary to include move Systems to RefDESI in the upgrade

There will be no automatic move of CYMWRLs into Design Reference databases, and Integrator no longer automatically creates a Link World. Project administrators are recommended to create a separate Design Reference database to hold links, and then use the new **Manage Links** dialogue, available from the **Integrator > Settings** menu or the **Compare/Update > Options** dialogue. This can be used to create and manage Link Worlds in the appropriate database, including consolidating links from separate databases.

1.6 Global Considerations

The following considerations must be made when applying upgrade parts to a Global project.

- The user must run an upgrade for all primary DBs at a location.
- For extracts, the entire extract family must be made primary at the same location
- System DBs should be upgraded at all locations.

2 Units

In earlier versions of Outfitting and other Dabacon-based AVEVA products the only physical dimension which was recognised in the storage of quantities was Length. Length is used for attributes of type DISTance and BORE, and the derived SQDI (squared length) and CUDI (cubic length) set via the UNIT field of an Attribute.

Most other Dabacon products had similar restrictions, except for Schematic data imported via Schematic Model Manager (refer to [Units in Schematic Model Manager](#)). Hull and Marine Drafting use the same unit handling as Tribon M3 and therefore require no upgrade or unit conversion.

For lengths the values are stored in the database in millimetres. Users can choose which length units are used in the GUI, from a predetermined set.

Overview of Units at 12.1

At 12.1 Outfitting and other Dabacon-based AVEVA products have been enhanced to recognise other dimensions which are relevant to attributes Engineers and Designers may want to use. How to create attributes with specific a specific dimension is described in 12.1 User Documentation.

The extra dimensions which have been introduced at 12.1 are managed in a similar manner to Lengths. There is a 'Database-Unit' for each dimension, in which the quantities will be stored, and a set of 'Display-Units' which the users can choose for their GUI. The dimensions and their Database Units are listed in 0.

Dimensions are now checked in calculations, so it is not possible to add a length quantity to a mass quantity.

Derived quantities are also recognised, so if a length (in millimetres) is divided by a time (in seconds) this is now recognised as a speed (in millimetres per second). These are also subject to dimension checking.

Prior to 12.1 users used standard attributes with dimensions, and may have created their own UDAs and catalogue properties which represent dimensioned quantities. It is expected that all of these will be attributes of type 'Real'.

Summary of Action to be Taken

To take advantage of the new functionality, attributes need to be set to the correct dimension. This has been done for the standard attributes. Users will need it to do it for their UDAs and catalogue and design parameters and properties. Any data imported to a Schematic database using Schematic Model Manager will need to have the 12.1 upgrade applied.

Users do not need to change all dimensions at the same time. For example Lengths are already handled correctly. It is expected that users will have stored angles in Degrees, so

they will also be handled correctly. It just required the administrator to identify which UDAs are angles and set their UUNIT to ANGL.

Separately for each of the dimensions listed in *Angles: - Unit Weights (per distance) (UMAS)* the administrator needs to determine

If all quantities have been stored in the new Database Units

- Set the UUNIT for any UDAs
- Any UDAs used to store the Unit values are no longer required and can be deleted
- Any user appware managing unit conversion or display can be removed or replaced by standard functions.

If all quantities have been stored in the same unit (which is not the new Database Unit)

- Set the UUNIT for any UDAs
- Output a datal file with the dimensions being set to numeric
UNITS NUMERIC TEMPERATURE
- Read the datal file back in with the current units set appropriately so that unqualified values are assumed to be in those units:
UNITS DEGF TEMPERATURE
- Any UDAs used to store the Unit values are no longer required and can be deleted
- Any user appware managing unit conversion or display can be removed or replaced by standard functions

If quantities have been stored in mixed units with a UDA recording the unit for each

- Set the UUNIT for any UDAs
- Set the dimensions to numeric
UNITS NUMERIC TEMPERATURE
- Output a file with the attribute values, with the value from the unit UDA appended
- Check the format of the value plus unit conforms to new input format rules
- If necessary edit the file with a text editor or script to achieve this
- Read the file back in
- Set current units as preferred
UNITS DEGF TEMPERATURE
- Any UDAs used to store the Unit values are no longer required and can be deleted
- Any user appware managing unit conversion or display can be removed or replaced by standard functions

If quantities have been stored in mixed units with 'custom and practice' being the only record of the unit

- It is hoped very few users are in this situation
- For the short-term set the dimensions to numeric
- Plan to move to more rigorous use of units, probably employing a combination of the techniques above.

2.1 Core Units (Outfitting)

At 12.1, Dabacon attributes will formally recognise all the dimensions listed in the table in *Dimensions and their Database Units*. The table also indicates the database unit which Dabacon will use from 12.1 onwards. Database units have been chosen to be that thought

to be the most commonly used unit. Where all quantities of a dimension are stored in the database unit, the new functionality can be used without any upgrade.

All attributes that have the UNIT field set for the first time, were until now stored as values with no specified unit. The units that were associated with their values in the past were determined by use and convention; and these could change from application to application, and project to project. This flexibility cannot be supported henceforth and a 'unit of storage' must be defined. AVEVA are setting the database units to those thought to be most commonly used in practice, but this will not be universally compatible. Hence the UNITS NUMERIC command is introduced to disable dimension conversion for selected dimensions.

If the 12.1 database unit does not agree with values stored in existing project databases, such data must be converted, or the units of measure of that physical dimension must be set to NUMERIC to disable dimension conversion for this dimension. Disabling a specific dimension in this way means that no advantages will be gained from the introduction of that dimension when working on the projects.

UNITS NUMERIC <dimension>

is used to suspend all default unit conversions on input and output for attributes of the nominated dimension.

- no conversion from the stored value will be made on output
- no unit qualifying strings will be appended to output values
- Input values with no qualifying unit strings will be stored without conversion in the database
- If input values have a unit qualifying string, then a conversion factor will be applied.

This is of particular value to users who wish to continue storing and using attribute values as now, and especially when the values stored are assumed by their system to be in units that are DIFFERENT to those now being assumed by the Outfitting or Dabacon system.

For the upgrade to 12.1 users will need to:-

Review all use of dimensions from the table below other than length

- In particular they will need to review their use of density and mass

For each dimension which has been used

- Are all stored quantities in the database unit?
- If not
 - Either set UNITS NUMERIC <dimension>
 - Or write a script to convert from their stored unit to database units and apply to all extracts of each DB used by the project. This will need to include Foreign DBs.

2.1.1 Dimensions of Standard Stored and Derived Attributes

Angles:

These attributes are assumed to stored be in Degrees

AALLAN	AANGXZ	AANGYZ	ACTANG	ADEG	ALLANG
ANGFR	ANGL	ANGLSP	ANGSPA	ANGSPB	ANGWL
AQAANG	AQANG	ASUB	BANG	BSCANG	CRCANG

DDEG	DEFSLO	DELANG	ENDA	FAAN	GANGLE
GRDDIR	HANGLE	INCL	KNUANG	LALLAN	LPITCH
LQAANG	LQANG	MATANG	MAXSLO	MINSLO	MINVER
NANGLE	OANG	ORIA	PALIG	PALLAN	PANG
PERS	PLAX	PPANFL	PPOFFT	PQAANG	PQANG
PXBS	PXTS	PYBS	PYTS	RANANG	SPMA
SPRA	STAN	TANGLE	TWSTAN	VANGLE	WCANG
WIANG	WRANG	XAMANG	XBSH	XINCL	XTSH
XXMANG	YBSH	YTSH			

Bores (BORE)

These attributes are assumed to stored be in mm. As in 12.0

ABOR	ACBO	ARRHEI	ARRWID	BORE	BOREAR
DPBO	DUCTHE	DUCTWI	HBOR	HEIARR	HHBO
HTBO	LBOR	LEAHEI	LEAWID	MAXB	NBORE
PBOR	PHBO	POBO	PPBO	PPHEI	PPWID
PTBO	SPRB	TBOR	WBORE	WIDARR	

Volumes (CUDI)

These attributes are assumed to stored be in mm³. As in 12.0 most of these are derived attributes

CMVOL	FLCVOL	FLLVOL	GVOL	HVOLU	MAXVOL
INVOL	NVOL	RVOL	SPMMVO	SPMNVO	

Currents (CURR)

These attributes are assumed to stored be in Amps

CURRENT

Densities (DENS)

These attributes are assumed to stored be in kg/m³

DENS	DNST	SPMDE
------	------	-------

Densities in Manufacturing Database (MAND)

These attributes are assumed to stored be in kg/mm³

MATDEN

Lengths and Distances (DIST)

These attributes are assumed to stored be in mm

As in 12.0. Too many to mention.

Voltages (EMF)

These attributes are assumed to stored be in Volts

VOLTAC VOLTDC

Forces (FORC)

These attributes are assumed to stored be in Newtons

EFORFLIMFORCSFOR

Resistances etc (IMPE)

These attributes are assumed to stored be in Ohms

IMPED REACT RESIS

Masses (aka weights) (MASS)

These attributes are assumed to stored be in Kg, As in 12.0, most of these are derived attributes

ASEWEI	BRIWEI	BRWEIG	BRWIWE	BRWWEI	CBWEIG
CIWE	CMFLW	CWEI	GWEI	HWEIG	MANWGH
NWEI	RWEI	SPMCWC	SPMCWS	SPMEWC	SPMEWS
SPMFLW	USCWEI	USCWWE	USRWEI	USRWWE	UWGHT

Content Density (PCUD)

These attributes are assumed to stored be in mm-3

CMCDE

Pressures (_PRES)

These attributes are assumed to stored be in Pascals

DPREMA	DPREMI	IPRE	MAXPRE	MINPRE	OPREMA
OPREMI	PRAV	PRES	PRMA	PRMI	RATI
RPRE	TPRESS	WPRE	YOUN		

Surface Density (PSQD)

These attributes are assumed to stored be in mm-2

INPIND PINDEN

Areas (SQDI)

These attributes are assumed to stored be in mm2. As in 12.0, most of these are derived attributes

BNDARE	BREARE	BRIARE	CBACXR	FLCARE	FLLARE
GAREA	GMOF	GSRF	HSRFA	INSARE	MAXARE
MINARE	NMOF	NSRF	PLAREA	RMOF	RSRF
SPMARA	SPMCAS	SPMCFA	SPMEAS	SPMRFA	UAREA
XAREA					

Temperatures (TEMP)

These attributes are assumed to stored be in degC

DTMPMA	DTMPMI	MAXTEM	MINTEN	OPTEMP	OTMPMA
OTMPMI	PTEN	RTEM	TEMP	TMAV	TMTA
TMMI					

Temperature Gradients (TPDI)

These attributes are assumed to stored be in degC/m

TGRA

Unit Weights (per distance) (UMAS)

These attributes are assumed to stored be in kg/m

UIWE UWEI

Derived from Local PTYP Attribute

These include Properties, catalogue answers, association real values etc.

Many of these are derived attributes produced from stored expressions. Care must be taken to make sure the result of the expression IS compatible with PTYP (i.e. of that physical dimension, or else purely numeric)

ANSW	CDPR	DDPR	DEPD	DEPR	FDEPD
FDEPR	FPRDE	FPROP	FTCDD	FTCDP	LDPR

MAXA	MAXMIN	PRDE	PROP	PROPRE	RDEP
REALEV	RPRO	TCDD	TCDP	TDPR	UMAX
UMIN					

Parameters Persisted from Actual Input Quantities

ADES	APAR	CPAR	DESP	IPAR	ODES
OPAR	PARA				

UDAS

UDAs have their database storage units determined by their UUNIT value which can be any of the dimension/(also known as UNIT) codes listed above (for example DIST, BORE etc.). UUNIT can also be a qualified unit value of the required dimension such as 1kg/m3 for density.

Expression Set Attributes

Many attributes including some of those listed above (for example property database attributes like UWEI), and also distance attributes in the catalogue, can be given expressions (algebraic and reverse polish). Care should be taken to make sure that the results of these expression are compatible with the dimension of the attribute (i.e. either of that physical quantity, or else purely numeric).

2.2 Dimensions and their Database Units

Name of Dimension	Database Units	Comment
AbsPressure	pascal	pressure may be absolute or gauge
Angle	degree	
AngularMomentum	N.m.s	
Area	mm2	
Bore	mm	Range of bore units limited to mm and inch (and Finch)
Capacitance	farad	
Charge	coulomb	
Conductance	siemens	
Content	mm-3	
Currency	USDollar	
Current	ampere	
Density	kg/m3	

Name of Dimension	Database Units	Comment
DensityMANDB	kg/mm3	densities stored in MANU database
ElectricConductivity	Si/m	
ElectricField	V/m2	
EMF	volt	
Energy	kiloWatthour	
EnergyDensity	kg/m3	
Force	newton	
FoulingFactor	m2.K/W	
Frequency	hertz	
GaugePressure	pascal	pressure may be absolute or gauge
HeatCapacity	J/m	
HeatingValue	J/m3	
HeatTransferCoeff	W/m2/K	
Impedance	ohm	
Inductance	henry	
Inertia	kg/m2	
KinematicViscosity	m2/s	
Length	millimetre	
LinearDensity	mm-1	
MagFieldIntensity	A/m	
MagFluxDensity	tesla	
MagneticFlux	weber	
Mass	kilogram	
MassFlow	kg/s	
Momentum	N.s	
Permeability	H/m	
Permittivity	F/m	
Power	kiloWatt	
Pressure	pascal	
RadiationDose	sievert	
Radioactivity	bequerel	
Resistivity	ohm/m	
RotationalStiffness	N.m/rad	

Name of Dimension	Database Units	Comment
SpecHeatCapacity	N/K	
SpecificEnergy	J/kg	
Speed	m/s	
Stiffness	N/m	
SurfaceDensity	mm-2	
Temperature	degCelsius	
TemperatureGradient	degC/mm	
ThermalConductivity	W/m/K	
ThermalResistance	K/W	
Time	second	
Torque	N.m	
UnitMass	kg/mm	
ViscosityDynamic	s/Pa	
Volume	mm3	
VolumetricFlow	m3/s	
None		numerical real attribute
WORD		used in assigning parameter dimensions etc.
Parameter		used for parameter attributes

2.3 Units in Project Data

2.3.1 Units in Pre-existing Attributes of Physical Quantity

The great majority of these are specific attributes of elements in the PROP (PROPCON) property database. Significant exceptions to these are some Temperature and Pressure attributes in other databases

In 12.1 these attributes will be assumed by the system to be stored in database units. This will not be a problem if this is, indeed, the case. It may not be a problem either if the user's use of the system and access of values does not make use of new 12.1 unit conversion and validation features.

It will be problem when conversions are being made in 12.1 and particularly if the database holds a mix of values stored in different units for the same physical quantity.

Non-significant Unit Definitions

The following attributes have had their UNIT field defined or modified in 12.1. These should not impact the end user since distances (and areas and volumes and bores) continue to be

processed as before, and Angles are unlikely to be stored in the database by users in any unit other than degrees.

Attribute	12.0 Unit	12.1 UNIT	Used in databases:
ADEG	NONE	ANGL	PADD
ANG	NONE	ANGL	DESI
ANGFR	NONE	ANGL	MANU
ANGLSP	NONE	ANGL	DESI
ANGSPA	NONE	ANGL	DESI
ANGSPB	NONE	ANGL	DESI
ANGWL	NONE	ANGL	MANU
ASUB	NONE	ANGL	PADD
BANG	NONE	ANGL	DESI
CRCANG	NONE	ANGL	MANU
DDEG	NONE	ANGL	PADD
DEFSLO	NONE	ANGL	DESI CATA
DELANG	NONE	ANGL	DESI
ENDA	NONE	ANGL	DESI
FAAN	NONE	ANGL	SYST
GANGLE	NONE	ANGL	DESI,PADD
HANGLE	NONE	ANGL	PADD
LPITCH	NONE	ANGL	DESI
MATANG	NONE	ANGL	DESI
MAXSLO	NONE	ANGL	DESI CATA
MINSLO	NONE	ANGL	DESI CATA
MINVER	NONE	ANGL	DESI CATA
NANGLE	NONE	ANGL	DESI
OANG	NONE	ANGL	PADD
ORIA	NONE	ANGL	DESI
PERS	NONE	ANGL	PADD
PPOFFT	NONE	ANGL	DESI
STAN	NONE	ANGL	DESI
TANGLE	NONE	ANGL	DESI
VANGLE	NONE	ANGL	DESI

Attribute	12.0 Unit	12.1 UNIT	Used in databases:
WCANG	NONE	ANGL	MANU
WIANG	NONE	ANGL	MANU
WRANG	NONE	ANGL	MANU
XBSH	NONE	ANGL	DESI
XTSH	NONE	ANGL	DESI
YBSH	NONE	ANGL	DESI
YTSH	NONE	ANGL	DESI
MAXVOL	NONE	CUDI	DESI
MINVOL	NONE	CUDI	DESI
AXSSIZ	NONE	DIST	PADD
INPIND	NONE	PSQD	DESI
PINDEN	NONE	PSQD	DESI
MAXARE	NONE	SQDI	DESI
MINARE	NONE	SQDI	DESI
PLAREA	NONE	SQDI	DESI MANU
SPMCAS	NONE	SQDI	DESI
SPMEAS	NONE	SQDI	DESI
SPMRFA	NONE	SQDI	DESI
UAREA	NONE	SQDI	MANU

More Significant Unit Definitions

The following stored attributes now have a defined physical dimension with associated unit of storage.

Some of these are more significant than others depending on the likelihood of end users having chosen to store values in units other than the database storage units:

The user should review how such attributes are stored in his databases, and convert/upgrade any values appropriately.

Attribute	12.0 unit	12.1 unit	Used in databases:
DENS	NONE	DENS	PROP
SPMDE	NONE	DENS	DESI
EFOR	NONE	FORC	DESI
FLIM	NONE	FORC	PROP
FORC	NONE	FORC	PROP
SFOR	NONE	FORC	DESI

Attribute	12.0 unit	12.1 unit	Used in databases:
MATDEN	NONE	MAND	MANU
ASEWEI	NONE	MASS	DESI
MANWGH	NONE	MASS	MANU
NWEI	NONE	MASS	DESI MANU
SPMCWC	NONE	MASS	DESI
SPMCWS	NONE	MASS	DESI
SPMEWC	NONE	MASS	DESI
SPMEWS	NONE	MASS	DESI
UWGHT	NONE	MASS	MANU
DPREMA	NONE	PRES	SCHE
DPREMI	NONE	PRES	SCHE
IPRE	NONE	PRES	PROP
MAXPRE	NONE	PRES	DESI
MINPRE	NONE	PRES	DESI
OPREMA	NONE	PRES	SCHE
OPREMI	NONE	PRES	SCHE
PRAV	NONE	PRES	DESI
PRES	NONE	PRES	DESI
PRMA	NONE	PRES	DESI
PRMI	NONE	PRES	DESI
RATI	NONE	PRES	CATA
RPRE	NONE	PRES	PROP
TPRESS	NONE	PRES	DESI PROP SCHE
WPRE	NONE	PRES	PROP
YOUN	NONE	PRES	PROP
DTMPMA	NONE	TEMP	SCHE
DTMPMI	NONE	TEMP	SCHE
MAXTEM	NONE	TEMP	DESI
MINTEM	NONE	TEMP	DESI
OPTEMP	NONE	TEMP	DESI
OTMPMA	NONE	TEMP	SCHE
OTMPMI	NONE	TEMP	SCHE
PTEM	NONE	TEMP	PROP

Attribute	12.0 unit	12.1 unit	Used in databases:
RTEM	NONE	TEMP	PROP
TEMP	NONE	TEMP	DESI PROP SCHE
TMMA	NONE	TEMP	DESI
TMMI	NONE	TEMP	DESI
TGRA	NONE	TPDI	PROP

Standard Units such as Temperatures and Pressures

A recommended method of upgrading data with such values is to output a datal file with the rogue dimensions being set to numeric, for example:

UNITS NUMERIC TEMPERATURE

This makes sure that such values are output without unit qualifiers.

These datal files can then be read back in, fully or partially (to overwrite rogue values) with the current units set appropriately so that unqualified values are assumed to be in those current units:

UNITS DEGF TEMPERATURE

Which makes sure that the correct conversion is made before storage (in this case to DEGC).

Unset temperatures and pressures will still be maintained with numerical values of -10000, and will not be converted. In fact any temperature less than absolute zero will be taken to be unset. If the user makes little use of temperatures or pressures apart from these unset values then no action need be taken.

Densities

Densities are particularly important as the system has always assumed that the value is in kg/m³ and made internal conversions to make sure mass calculations of steelwork from its computed geometric volume where returned as kg. Some users may have taken advantage of this and stored densities in lb/m³ to make sure masses were returned in imperial pounds.

2.3.2 Attributes Stored as Expressions

The upgrade procedure above will not necessarily work for attributes that are stored and output as expressions as the text actually input. Thus if input without units then the output will always be generated and stored without units and will be interpreted as a value in current units when the expression is evaluated for example, such a temperature will be output as (180).

And so should always be evaluated with current units as degF (if degF is the current unit) - which will probably be the correct interpretation most of the time in practice. However to make sure full consistency whatever current units are in place the expression must be edited to something like:

(180degF)

To avoid any ambiguity.

The same principles apply (and the above advice should have been followed for good practice) for any similar distance attributes.

Specific and very common examples of this are the many attributes in the geometrical section of the catalogue which are stored as expressions that resolve to distances (heights, radii, diameters etc.)

This is not (in 12.1), and never has been, a problem generally as such expressions in the catalogue are ALWAYS EVALUATED WITH CURRENT UNITS SUSPENDED, and unqualified values are therefore assumed to be in database units mm. This is not the case in the properties database.

A list of attributes stored as expressions with new or modified physical dimensions are:

Attribute	12.0 unit	12.1 unit	Used in databases:
ALLANG	NONE	ANGL	CATA
PANG	NONE	ANGL	CATA
PLATYP	NONE	ANGL	CATA
PXBS	NONE	ANGL	CATA
PXTS	NONE	ANGL	CATA
PYBS	NONE	ANGL	CATA
PYTS	NONE	ANGL	CATA
ACBO	NONE	BORE	PROP,CATA
PBOR	NONE	BORE	CATA
BDIA	NONE	DIST	CATA
BTHK	NONE	DIST	CATA PROP
BTOL	NONE	DIST	CATA PROP
CORA	NONE	DIST	CATA PROP
DRAD	NONE	DIST	CATA
DWID	NONE	DIST	CATA
DX	NONE	DIST	CATA
DXL	NONE	DIST	CATA
DY	NONE	DIST	CATA
DYL	NONE	DIST	CATA
GAPALL	NONE	DIST	PROP CATA
MINBEN	NONE	DIST	PROP,CATA
OUTD	NONE	DIST	CATA,PROP
OUTSD	NONE	DIST	CATA,PROP
PBBT	NONE	DIST	CATA
PBDI	NONE	DIST	CATA
PBDM	NONE	DIST	CATA

Attribute	12.0 unit	12.1 unit	Used in databases:
PBOF	NONE	DIST	CATA
PBTP	NONE	DIST	CATA
PCBT	NONE	DIST	CATA
PCOF	NONE	DIST	CATA
PCTP	NONE	DIST	CATA
PDIA	NONE	DIST	CATA
PDIS	NONE	DIST	CATA
PHEI	NONE	DIST	CATA
POFF	NONE	DIST	CATA
PRAD	NONE	DIST	CATA
PTCPOS	NONE	DIST	CATA
PTDI	NONE	DIST	CATA
PTDM	NONE	DIST	CATA
PTEPOS	NONE	DIST	CATA
PTSPOS	NONE	DIST	CATA
PWID	NONE	DIST	CATA
PX	NONE	DIST	CATA
PXLE	NONE	DIST	CATA
PY	NONE	DIST	CATA
PYLE	NONE	DIST	CATA
PZ	NONE	DIST	CATA
PZLE	NONE	DIST	CATA
WTOL	NONE	DIST	PROP CATA
XAREA	NONE	SQDI	PROP CATA
BTYP	NONE	WORD	CATA
PCON	NONE	WORD	CATA
Expression Attributes which should be reviewed for 12.1			
CURREN	NONE	CURR	PROP CATA
VOLTAC	NONE	EMF	PROP CATA
VOLTDC	NONE	EMF	PROP CATA
IMPED	NONE	IMPE	PROP,CATA
REACT	NONE	IMPE	PROP CATA
RESIS	NONE	IMPE	PROP CATA

Attribute	12.0 unit	12.1 unit	Used in databases:
CWEI	NONE	MASS	PROP,CATA
CWEI	NONE	MASS	PROP,CATA
USRWEI	NONE	MASS	DESI
USRWWE	NONE	MASS	DESI
UIWE	NONE	UMAS	PROP CATA
UWEI	NONE	UMAS	PROP CATA

Other Uses of Expressions in Project Data

The same principles apply to other uses of expressions in projects -

- associations,
- Outfitting Draft representation rules,
- collections,
- auto routing,
- auto colour,
- attribute rules

It is probably unlikely that any existing rules will make any substantial use of attributes that now have new physical quantities assigned, or that rely on specific values in specific units. However if they do then users must make sure that the expressions are dimensionally consistent, robust, and can survive current unit changes.

2.3.3 Units in Catalogue and Design Properties

Users may have created properties that represent a physical quantity and so should have a dimension assigned. The method of doing this is through its PTYPE. In the past this could not be stored, except for distances, bores and none.

The PTYPE persists the physical dimension of the property. If this was DIST or BORE then the results were distances, and are now checked to either be distances, or if purely numeric then taken to be a distance in current units.

If the PTYPE was NONE then the result should be purely numeric. If it does have dimension then a warning or error will be issued on evaluation.

If the PTYPE is unset, or is DATA then the result will have the dimension of whatever the expression of the property evaluates to. This may be evaluated to a different physical quantity in 12.1 since expressions accessing attribute values will be impacted by the dimension of such attributes.

Expressions will track the resulting physical quantity. For example if converting a density to a mass (commonly termed weight) then it is not good enough to multiply it by the value of the volume of material for example:

$DENS * 100 * 50 * 2500$

This will simply produce another, different density. The density must be multiplied by values that compute to a volume, for example:

$DENS * XLEN * GSRF$

Attributes that have the physical quantity of their values defined by another attribute (normally PTYP) are:

Attribute	12.0 unit	12.1 unit	Used in databases:
ANSW	NONE	PTYP	CATA
MAXA	NONE	PTYP	CATA
MAXMIN	NONE	PTYP	DESI
UMAX	NONE	PTYP	DICT
UMIN	NONE	PTYP	DICT
Expressions similarly controlled are:			
DDDF	NONE	PTYP	DESI
DDPR	NONE	PTYP	DESI
DPRO	NONE	PTYP	CATA
PPRO	NONE	PTYP	CATA
REALXP	NONE	PTYP	DESI
Derived attributes that present such values to the user are:			
CDPR	NONE	PTYP	DESI
DEPD	NONE	PTYP	DESI
DEPR	NONE	PTYP	DESI
LDPR	NONE	PTYP	DESI
PRDE	NONE	PTYP	DESI
PROP	NONE	PTYP	DESI
PROPRE	NONE	PTYP	DESI
RDEP	NONE	PTYP	DESI
REALEV	NONE	PTYP	DESI
RPRO	NONE	PTYP	CATA
TCDD	NONE	PTYP	DESI
TCDP	NONE	PTYP	DESI
TDPR	NONE	PTYP	DESI

If the user has made extensive use of design properties and other typed expressions, such as in associations, or in the property database, or in the catalogue he should check that they are dimensionally robust.

2.3.4 Units in Catalogue and Design Parameters

Up to and including 12.0 all values in the catalogue and design parameters were simply numbers without physical quantity. If they were distances then values should be entered in as mm to make sure that catalogue expressions evaluated correctly.

In 12.1 this is no longer necessary. If the user enters parameters with a unit qualifier then this determines the physical dimension of that parameter. Such parameters are always stored in database units of their physical dimension. The physical dimension persists until redefined, and impacts any expressions in which the parameter is used.

For such parameters the pseudo attributes that return word and current distance values of parameters are obsolete and unnecessary as the parameter is known to be a distance or a word.

However the existing behaviour of un-dimensioned parameters persists in 12.1 and there is no immediate need to upgrade existing data.

Users' appware, however may need to be reviewed for dimensional robustness once dimensioned parameters appear in a project.

Stored parameters maintaining this behaviour are:

Attribute	12.0 unit	12.1 unit	Used in databases:
DESP	NONE	UNIPAR	DESI
IPAR	NONE	UNIPAR	DESI
PARA	NONE	UNIPAR	CATA, PADD
Pseudo (derived) attributes presenting these values are:			
ADES	NONE	UNIPAR	DESI
APAR	NONE	UNIPAR	DESI
CPAR	NONE	UNIPAR	DESI
ODES	NONE	UNIPAR	DESI
OPAR	NONE	UNIPAR	DESI

2.3.5 Derived Attributes

Many derived attributes now have updated physical quantities. This could impact any appware that uses these attributes, but is not relevant for updating existing projects. These are:

Attribute	12.0 unit	12.1 unit	Used in databases:
AALLAN	NONE	ANGL	DESI
AANGXZ	NONE	ANGL	DESI
AANGYZ	NONE	ANGL	DESI
ACTANG	NONE	ANGL	DESI
AQAANG	NONE	ANGL	DESI
AQANG	NONE	ANGL	DESI
BSCANG	NONE	ANGL	DESI
GRDDIR	NONE	ANGL	DESI
LALLAN	NONE	ANGL	DESI

Attribute	12.0 unit	12.1 unit	Used in databases:
LQAANG	NONE	ANGL	DESI
LQANG	NONE	ANGL	DESI
PALIG	NONE	ANGL	DESI
PALLAN	NONE	ANGL	DESI
PPANFL	NONE	ANGL	DESI
PQAANG	NONE	ANGL	DESI
PQANG	NONE	ANGL	DESI
RANANG	NONE	ANGL	DESI
SPMA	NONE	ANGL	DESI
SPRA	NONE	ANGL	DESI
TWSTAN	NONE	ANGL	DESI
XAMANG	NONE	ANGL	DESI
XINCL	NONE	ANGL	DESI
XXMANG	NONE	ANGL	DESI
SPMMVO	NONE	CUDI	DESI
SPMNVO	NONE	CUDI	DESI
CMCDE	NONE	PCUD	DESI
CBACXR	NONE	SQDI	DESI
GAREA	DIST	SQDI	DESI
SPMARA	NONE	SQDI	DESI
SPMCFA	NONE	SQDI	DESI
DNST	NONE	DENS	DESI,CATA
BRIWEI	NONE	MASS	DESI
BRWEIG	NONE	MASS	DESI
BRWIWE	NONE	MASS	DESI
BRWWEI	NONE	MASS	DESI
CBWEIG	NONE	MASS	DESI
GWEI	NONE	MASS	DESI
RWEI	NONE	MASS	DESI
SPMFLW	NONE	MASS	DESI
USCWEI	NONE	MASS	DESI
USCWWE	NONE	MASS	DESI

Some already had the correct dimension, most distances and bores, and many volumes and areas.

2.4 Units in Datal and Output Files

Output files are now output with all values of physical quantity output with unit qualifiers.

This makes sure that such files can be input back into a system without making sure that a compatible set of current units are established before entry.

2.5 Units in Specon and Spec Files

Spec files to be output with unit qualified values whenever possible

Unit qualified input to be read, and unit qualifiers to determine PTYP of answers.

PTYP of ANSWs to be deduced from unit qualified input, and from standard questions such as PBOR, TEMP, PRES etc. which the system already identify as physical quantities.

2.6 Units and Appware

This section describes the impact of the 12.1 Units development on PML code, and it describes PML functions provided to handle common operations with units in 12.1. The core Units changes have been implemented so that the impact on PML code is minimised, but some changes to PML code are inevitable.

This section covers:

- PML coding scenarios that may cause problems with Units functions in 12.1
- Functions that have been provided to help make 'units-safe' PML applications in 12.1.

2.6.1 A Very Brief Introduction to Units by Example

In order to understand how the Units changes affect PML code, the PML writer needs to understand how REAL numbers and PML expressions behave in 12.1. This section illustrates use of new core units functions with a few simple command line examples.

Look at the effect of setting MASS units, using mass unit qualifiers (kg), and using new methods available on REAL objects. Notice that the real variables !m and !p know that they represent a MASS, and that the value stored in the variable !p is automatically converted from kilograms to the current working unit.

```
!unitObject = object unit('kg')
!massObject = object measure('mass')
!massObject.setunits(!unitObject)

!m = 1kg
Q VAR !m
<REAL> 1kg
Q VAR !m.string()
<STRING> '1kg'
```

```
$P $!m
1kg
Q VAR !m.units()
<UNIT> kilogram
Q VAR !m.dimension()
<MEASURE> Mass
-- Now look at the value 1 kg with current working MASS units
set to Pounds
!unitObject = object unit('pound')
!massObject.setunits(!unitObject)
!p = 1kg
Q VAR !p
<REAL> 2.20462262184878lb
Q VAR !p.string()
<STRING> '2.20462262184878lb'
Q VAR !p.units()
<UNIT> pound
```

Go to a BOX element in the database to see area and volume units being derived from PML calculations:

```
q var !!ce.xlen
<REAL> 510mm
!area = !!ce.xlen * !!ce.ylen
!volume = !area * !!ce.zlen
q var !area !volume
<REAL> 102000mm2
<REAL> 23460000mm3
q var !!ce.gvol
<REAL> 23460000mm3
Q VAR !area.units() !area.dimension()
<UNIT> mm2
<MEASURE> Area
```

Go to a SCTN element with a MATREF set to see a compound unit derived from mass and distance:

```
UNITS METRE DIST
q var !!ce.gweight
<REAL> 17.794kg
q var !!ce.cutlength
<REAL> 0.774996172710133metre
!unitWeight = !!ce.gweight / !!ce.cutlength
```

```
q var !unitWeight
<REAL> 22.959536446628kg/m
Q VAR !unitWeight.units() !unitWeight.dimension()
<UNIT> kg/m
<MEASURE> UnitMass
```

2.6.2 Current Working Units and FORMAT Objects

As a PML developer, it is important to understand the difference between current working units and displayed units.

It is not always necessary to change current working units to provide input or generate output in a given unit. Changing current working units can be difficult to manage in PML code. Care has to be taken when saving current unit settings and then restoring them when an operation is complete.

As an example, the following extract of PML code shows that an area can be output in square metres regardless of the current distance units.

```
-- Construct FORMAT object for area output in square metres
!sqmAreaFormat = object FORMAT()
!sqmAreaFormat.dimension = 'SQDI'
!sqmAreaFormat.units = 'metre2'
!sqmAreaFormat.dp = 2
!sqmAreaFormat.label = 'UNITS'
q var !!ce.nsrfs.string()
<STRING> '67402853.2666297mm2'
q var !!ce.nsrfs.string(!sqmAreaFormat)
<STRING> '67.40metre2'
```

2.6.3 What to look out for in PML Code

Distance Units

PML code has evolved to solve problems with existing distance units in Outfitting. Most of this code has been implemented to allow Outfitting to present itself correctly in metric and imperial distance units. The techniques used by PML developers to present data in the correct units are varied, so it is difficult to describe every case where code may need to be changed to work well in 12.1.

There are a few Outfitting functions that require all values to be specified in millimetres (the database storage unit for distance). PML code has to protect users working with imperial distances from these functions by switching units to MM, executing the function with values in mm, and then switching back to saved working units. Old techniques used for switching units do not work with new distance units.

Most PML code assumes that the only metric measure of distance is millimetres. Now the current distance units can be set to other metric units such as centimetres or metres, and imperial distance units can be set to decimal feet or yards. So, it is now necessary for PML

code to protect users working in centimetres or metres from functions and data that work only in millimetres.

Area and Volume

Before 12.1, PML code had to convert the result of an area or volume query (i.e. NSRF or NVOL) to the required units. This is now done by the core so no unit conversion calculations are necessary in PML.

However, it will be necessary to replace PML code that converts the value returned by area or volume queries to another unit (for example from cubic mm to cubic metres). Otherwise area and volume conversions may be done twice - firstly by the core, and then by PML.

New Dimensions

New issues and new opportunities arise with real values stored in Outfitting databases that previously had no physical dimension associated with them. This includes angle, mass, pressure, density, temperature and the electrical quantities added at Outfitting 12.0 for the cabling application.

The system assumes that all values stored in database attributes that were previously undimensioned are stored in database units, for example Degrees Centigrade for temperature, Pascal for pressure, kg for mass. However, there was nothing preventing users from storing these properties in other units. Some users have stored temperature in Fahrenheit and mass in pounds, and worse still, they might have stored mixed unit values for the same dimension in the same Project (for example some temperatures in Fahrenheit and others in Celsius).

As a PML developer, you need to know that values retrieved from temperature, pressure, mass, density and angle fields in the database will be assumed to have been stored in database units and will be converted automatically into the current working units for that dimension.

For example, a value 212 stored in a temperature attribute before 12.1 will be interpreted as 212degC or 413.6degF when it is retrieved from the database.

Angles

The database unit for angle properties is degrees. No other current angle working angle unit can be used, but using FORMAT objects it is possible to input and output angles in radians, grads etc.

Design and Catalogue Parameters

Dimensions of Design and Catalogue parameters have not been stored until 12.1. Even parameters representing a distance can only be identified if they are accessed using a DIST data property in a Dataset.

In 12.1, the issue faced by PML developers is that parameter dimensions can be specified when they are updated in the database, but there is no requirement to force all parameters to be upgraded. This means that when directly accessing a parameter value (not using a DATA Property) the result returned could be an undimensioned REAL value assumed to be in database units or it could be a dimensioned value that will be returned in the current working units for that dimension. A new PML function is provided to help deal with this issue.

Rounding Values

Occasionally values are rounded up, down or to the nearest integer value in PML code. For imperial distances, there may be the requirement to round values to the nearest 1/32nd

inch. This has been achieved in various ways, for example using `int()` and `nint()` functions, using `FORMAT` objects with the `.DP` property set to 0 or `.DENOMINATOR` property set to 32, or by using the `Real` object `.string('D0')` function. This is dangerous where the code incorrectly assumes that the current value is in MM.

The following code would probably have an undesired result.

```
UNITS METRES DIST
!distance = 123.45678mm
!displayedDistance = !distance.string('D0') or
!displayedDistance = !distance.int().string()
The result would be    <STRING> '0'
Not 123mm or 0.123 metres
```

2.6.4 Units Qualifiers

At 12.1, unit qualifiers are output in most cases where a value is converted to a string. This was not the case in 12.0. For example:

Where `!d = 12mm`

Command	Pre-12.1 Result	12.1 Result
<code>Q var !d.string()</code>	<code><STRING> '12'</code>	<code>'<STRING> '12mm'</code>
<code>\$P \$!d</code>	12	12mm

Command output (DATAL) files now have unit qualifiers on all united values, which mean that there are fewer problems to resolve when loading into an imperial units project a DATAL file that was produced in a project with mm distance units.

The PML writer needs to be aware that pre-12.1 code as follows will need to be changed:

```
!dist    = 12mm
!value = !dist.string() + 'mm'
Q var !value
```

Before 12.1 the result would be:

```
<STRING> '12mm'
```

At 12.1 the result will be:

```
<STRING> '12mmmm'
```

At 12.1, the required result is achieved with the simpler expression:

```
!value = !dist.string()
```

2.6.5 Testing for Metric or Imperial Distance and Bore Units

There are several methods used in old PML code to find out whether the current units are metric or imperial. These methods typically parse the result of the command `VAR !units` which returns a string like `INCH Bore INCH Distance`

This technique will not work in 12.1 for any current distance units other than mm or inch. Code that tests for imperial or metric units must be replaced by the new `!!isImperialLength` PML function or by a PML UNITS object method.

Old Code Samples	Replacement Code
var !units UNITS !metric = (!units.part(3) eq 'MM')	!metric = !!isImperialLength('DIST').not()
var !units SPLIT UNITS if (!units[3] neq MM) then	if (!!isImperialLength('DIST')) then
var !units SPLIT UNITS if (!units[1] neq MM) then	if (!!isImperialLength('BORE')) then
var !units units !imp = (!units.split()[3].neq('MM'))	!dm = object MEASURE('DIST') !imp = !dm.currentUnits().isImperial()

An example of testing a real variable using the new PML Units object:

```
!metric = !realVariable.dimension().currentUnits().isMetric()
```

2.6.6 Save and Restore Units

Before 12.1, the most commonly used methods to save and restore units are:

```
var !units units
```

```
mm DISTANCE
```

```
... Code that must be executed in MM distance
```

```
--reset units
```

```
$!units
```

If the current distance unit is Metres or Centimetres, this code will not revert back to the original distance units. The command `$!units` will execute the command `MM DIST MM BORE` leaving current distance units as MM.

Old PML save and restore units code must be replaced by the new PML COMUNITS object or the new core UNITS SUSPEND functions.

Old Code Samples	Replacement Code
var !units units mm BORE mm DISTANCE Code that must be executed in MM --reset units \$!units	!savedUnits = object COMUNITS(true) UNITS mm BORE UNITS mm DISTANCE ... Code that must be executed in MM --reset units !savedUnits.restoreSavedUnitsByPtype('DIST') !savedUnits.restoreSavedUnitsByPtype('BORE')

An alternative method of saving and restoring units is to use the following methods on the MEASURE object, which are described in the Software Customisation Reference manual:

.suspend()

.unSuspend()

.reinstate()

Or use the equivalent UNITS command:

UNITS SUSPEND

.....

UNITS UNSUSPEND, or UNITS REINSTATE

All current units of ALL DIMENSIONS simultaneously can be suspended and operation reverts to purely database units (i.e. mm, kg, degC etc.) using the commands UNITS SUSPEND, UNITS UNSUSPEND (by a single previous suspend), or UNITS REINSTATE to pop all previous suspends.

Or by PML methods on a measure object:

!measure.suspend(),

!measure.unsuspend(),

!measure.reinstate()

This can avoid the need to save previous units entirely.

However, note that if any current units are set during a suspension, then this setting will be ignored until the unsuspension, at which point the change will become active.

2.6.7 Units Conversions

There are several methods used to convert real numbers to distance values in old PML code. For example, taking a catalogue or design parameter value which is known to be a distance in millimetres and converting it to a distance value in current distance units.

One of the most commonly used methods is to convert a number to a string, append 'mm' to the string, and evaluate the string back to a REAL value. This will not work at 12.1.

Some old PML code converts between mm and inch by dividing or multiplying by 25.4. This will not work at 12.1 because current distance units could be cm, metres, feet etc.

Another common requirement is to convert a value in current distance units to a value in millimetres for core functions that only accept values in MM.

New PML functions and new methods on REAL numbers have been provided to help with units conversions.

Old Code Samples	Replacement Code
Converting an undimensioned value to MM: <pre>!val = !!ce.desp[1] !stringVal = !val.string() + 'mm' !distVal = !stringVal.real()</pre>	NOTE: Parameters might be numeric or dimensioned in 12.1. This code assumes that the numeric (undimensioned) values are in mm. <pre>!distVal = !!comConvertUnknownValue(!!ce.desp[1], 'MM') Or !distVal = !!ce.desp[1].convertunits('mm')</pre>
<pre>!width = !sctn.catref.para[2]</pre>	<pre>!width = !!comConvertUnknownValue(!sctn.catref.para[2], 'MM') Or !width = !sctn.catref.para[2].convertunits('mm')</pre>
Converts current distance units to MM: <pre>!format = object FORMAT() !format.units = 'MM' !format.dimension = 'L' -- Get volume box of item !volume = object VOLUME(!element) -- Set view limits (must be mm) !array[1] = !volume.from.east.string(!format).real()</pre>	<pre>-- Get volume box of item !volume = object VOLUME(!element) -- Load view limits with Volume !array[1] = !!comValueConvert(!volume.from.east, 'MM') Or !array[1] = !volume.from.east.convertunits('MM')) .convertunits() result will have the required units. Conversion is from the original units of the variable. If the variable has no units then database storage units are assumed (i.e. mm for distance)</pre>
<pre>!dist = object BLOCK(!width & 'mm') !width = !dist.evaluate()</pre>	<pre>!width= !!comConvertUnknownValue(!width, 'MM') Or !width = !width.convertunits('mm')</pre>
Convert MM value to INCH: <pre>!v = !vMetric / 25.4</pre>	<pre>!vInch = !!comUnitsConvert(!vMetric, 'MM', 'INCH') Or !vInch = !vMetric.cast(object unit('mm')).convertunits('inch') Note: The cast to mm is required because convert units will assume database units for numeric values, which are mm. If the original value is not in mm (for example cm) then the cast is necessary..</pre>

Old Code Samples	Replacement Code
<pre>if (!isMetric) then !arrowHeight = 100 else !arrowHeight = 4 * 25.4 endif</pre>	<pre>!arrowHeight = 100mm</pre> This does not give the same result in imperial, but the difference between 100mm and 101.6mm arrow height on an aid arrow is insignificant. If current units are cm then the old code would set the arrow height to 100cm!
<pre>Converting area to metre2 or feet2: !area = !ce.nsr var !units split units if (!units[3] neq 'MM') then -- convert to mm to metre squared !area = !area / 1000 / 1000 else -- convert inch to foot squared !area = !area / 12 / 12 endif</pre>	<pre>!area = !ce.nsr</pre> No conversion is necessary. Output in other units can be handled using FORMAT objects

2.6.8 Remove Units from a REAL

Sometimes it is necessary to work with a real value without units. A core method on REAL is provided for this.

```
!val = 123.5mm
!r = !val.value()
Q var !r
<REAL> 123.5
```

2.6.9 Units Display

Display of values with or without unit qualifiers is mostly controlled by using FORMAT objects, particularly !!distanceFmt. This is still OK in 12.1. The REAL.string() method now returns a STRING value with unit qualifier.

Old Code Samples	Replacement Code
<pre>!output.append('FIRSTGAP: ' + !this.firstGap.val.string() + ' ' + !unit)</pre>	<pre>!output.append('FIRSTGAP: ' + !this.firstGap.val.string())</pre>

2.6.10 Text Boxes on Forms

The main impact on PML forms will be seen on text boxes. Instead of these holding the value as a number they will now often be physical quantities (most frequently distance, but also angle, mass, area, volume, pressure, temperature). When these are populated by the system, especially with a FORMAT object, they will have their current working units attached.

This means that the width of some input fields on forms must be increased to allow for the unit qualifier.

ISOU text boxes normally found on old PML 1 forms will be parsed, and in 12.1 all forms of distance will be accepted (there was only limited parsing of ISOU text boxes prior to 12.1). Some old forms in the standard product used ISOU text fields for values that were not distances. This was an error, but it usually went unnoticed. At 12.1 an error is issued if an ISOU field is used incorrectly. Note that ISOU text gadgets are deprecated and documentation of how to create them has been removed, but the functionality has not been removed from the product.

Files written for output and for configuration will have units appended (mainly because the .string() method and \$! and var ! commands will all generate strings with units attached. If this is not wanted then .value() must be used first remove the unit entirely by making the number purely numeric.

2.6.11 Dimension and Units of REAL Expressions

The DIMWORD function has been provided to test the dimension of REAL expressions to validate that an expression delivers the expected type of result. DIMWORD returns the PTYPE of the dimension of an expression.

For example,

```
Q DIMWORD ( 1 KG PER CU METRE )
DENS
```

```
Q DIMWORD (2 * pi * power(100mm,2))
SQDI
```

```
Q DIMWORD( gweight / cutlength )
UMAS
```

A more descriptive name of the dimension of an expression can be returned by using the DIMENSIONOF function:

```
Q DIMENSIONOF (1 kg/m3 )
Density
```

The unit the evaluation (i.e. current units of the dimension) as unit qualifier is returned by the UNITSOFF function:

```
Q UNITSOFF( GVOL * DNST )
kg
```

2.6.12 Other Units Considerations

There are some cases in old PML code where positions were constructed as follows:

```
!x = 100mm
!y = 200mm
!z = 300mm
!pos = object POSITION('E' + !x.string() + 'N' + !y.string() + 'U' +
!z.string() + 'WRT WORLD')
```

Or

```
!pos = object POSITION('E' & !x & 'N' & !y & 'U' & !z & 'WRT WORLD')
```

These expressions will generate an error because the strings would have evaluated to

E100N200U300WRT WORLD before 12.1 - This is valid syntax.

But at 12.1 the string evaluates to

E100mmN200mmU300mmWRT WORLD - which is not valid syntax.

Make sure that there is a space between the real value and the next command word.

2.6.13 Display Units

Display of values with unit qualifiers and control of input and output for Forms & Menus fields is determined by using the PML FORMAT object. The PML FORMAT object deals with presentation of dimensioned real values - it does not change current working units.

Existing Format objects will operate as before, although you should be careful about use of Format objects to round values to a given number of decimal places or to a given fraction of an inch. Distance and Bore formats work as before.

There is a new PML Format object facility that can be very useful. It is possible to set the .LABEL property to 'UNITS' and leave the .UNITS field unset which means that a value processed with this Format object will appear in current working units with its unit qualifier.

A modified !!COMFORMATS object establishes a new set of global units Format variables as follows:

Physical Dimension of Quantity	PTYPE UNIT field value	Global Format Object
Distance (Length)	DIST	!!distanceFmt
Bore (Length)	BORE	!!boreFmt
Area	SQDI	!!areaFmt
Volume	CUDI	!!volumeFmt
Linear Density	PDIS	NONE
Surface Density	PSQD	NONE
Content Density	PCUD	NONE
Angle	ANGL	!!angleFmt
Mass	MASS	!!massFmt
Unit Weight (mass per unit length)	UMAS	!!unitMassFmt
Density (in PROP db)	DENS	!!densityFmt
Density (in MANU db)	MAND	NONE
Temperature gradient (temperature per unit length)	TPDI	!!temperatureFmt
Pressure	PRES	!!pressureFmt
Current	CURR	!!currentFmt

Physical Dimension of Quantity	PTYPE UNIT field value	Global Format Object
EMF (i.e. Voltage)	EMF	!!emfFmt
Impedance	IMPE	!!impedanceFmt
Time	TIME	!!timeFmt
Force	FORC	!!forceFmt
Energy	ENER	!!energyFmt
Power	POWE	!!powerFmt

!!COMFORMATS does not support user defined formats for any dimension other than distance and bore.

All of the new global format objects except for ANGLE track the current working units, so the display units are tied to the working units, but this may change in future.

2.6.14 New and Modified Units PML Functions

Save and Restore Units

Object	COMUNITS
Description	Handles current working units for PML code.

Methods

Name	Result	Description
.comUnits()	COMUNITS	Object constructor - does not save the current working units for all base dimensions
.comUnits(BOOLEAN)	COMUNITS	Object constructor - Saves the current working units for all base dimensions in argument is TRUE. Current units not saved if argument is FALSE.
.getCurrentUnitsByPtype (STRING)	STRING	The current working unit for the given PTYPE is returned.
.isImperialSavedBore()	BOOLEAN	True if saved (not current) bore units are imperial
.isImperialSavedDist()	BOOLEAN	True if saved (not current) distance units are imperial
.isImperialSavedLength(STRING)	BOOLEAN	True if saved (not current) units defined by the argument ('DIST' or 'BORE') are imperial
.restoreSavedBoreDist()		Restores DIST and BORE working units to last saved units. Equivalent to .restoreSavedUnitsByPtype('BORE') .restoreSavedUnitsByPtype('DIST')

Name	Result	Description
.restoreSavedUnitsByPt ype (STRING)	STRING	The saved current working unit is restored for the given PTYPE. The restored working unit for the given PTYPE is returned.
.saveCurrentUnits()	None	Saves the current working units for all base dimensions
.setUnits(STRING, STRING, STRING)	STRING	Sets current units for a given dimension. First argument - unused - pass an empty string Second argument - dimension PTYPE Third argument - Unit qualifier for new current unit - must be consistent with PTYPE Return value if the unit qualifier of the current unit for the given PTYPE (in case of error).

Test for Metric or Imperial Distance and Bore Units

Function	!!isImperialLength(STRING) is BOOLEAN		
Description	Returns True if given distance unit type is imperial, False for metric for example true if DIST unit is INCH, FINCH, FOOT, YARD etc.		
Arguments	1	STRING	'DIST' or 'BORE'

Unit Conversions - PML Functions

Function	!!comUnitsConvert(REAL, STRING, STRING) is REAL		
Description	Returns undimensioned real value for the conversion of the input REAL value and unit to the output unit, for example: !!comUnitsConvert(2, 'INCH', 'MM') returns 50.8 !!comUnitsConvert(2.54, 'CM', 'DIST') returns 1 where current DIST unit is INCH !!comUnitsConvert(1, 'kg', 'lb') returns 2.20462262184878 !!comUnitsConvert(1, 'degC', 'degF') returns 33.8 If the conversion fails for any reason, the original input value is returned.		

Arguments	1	REAL	REAL value to be converted from input unit to output unit. Dimension of input REAL is ignored.
	2	STRING	Input unit or 'DIST' or 'BORE'. Valid unit qualifier. If 'DIST' or 'BORE' then current unit for DIST or BORE is used.
	3	STRING	Output unit or 'DIST' or 'BORE'. Valid unit qualifier. If 'DIST' or 'BORE' then current unit for DIST or BORE is used.

Function	!!comValueConvert(REAL, STRING) is REAL		
Description	Returns undimensioned real value for the conversion of the input REAL value to the output unit. If REAL is not dimensioned it is assumed to be a value in the requested unit, so the same value is returned. If REAL is dimensioned, an undimensioned REAL value is returned representing the conversion of the dimensioned input value to the output unit, for example: !!comValueConvert(2, 'MM') returns 2 !!comValueConvert(2cm, 'MM') returns 20 !! comValueConvert (1kg, 'lb') returns 2.20462262184878 !! comValueConvert (1degC, 'fahrenheit') returns 33.8 If the conversion fails for any reason, the original input value is returned. The dimension of the input REAL must be consistent with the requested output unit.		
Arguments	1	REAL	REAL value to be converted to the specified output unit.
	2	STRING	Output unit or 'DIST' or 'BORE'. Valid unit qualifier. If 'DIST' or 'BORE' then current unit for DIST or BORE is used.

Function	!!comConvertUnknownValue(REAL, STRING) is REAL		
Description	Imposes default units on undimensioned REAL values. If the REAL value is undimensioned, the specified unit is applied to the REAL value. If the REAL value is dimensioned, the specified default unit is ignored and the REAL value is returned unchanged. This function is provided to deal with Design and Catalogue parameters that may have been stored as dimensioned or undimensioned values. It is mostly used to apply database default units to undimensioned values for example: If current distance unit is INCH: • DESP[1] = 25.4 !! comConvertUnknownValue(!ce.DESP[1], 'MM') returns 1 inch • DESP[1] = 1 inch !! comConvertUnknownValue(!ce.DESP[1], 'MM') returns 1 inch If current temperature units are Fahrenheit: • PARA[8] = 1 !! comConvertUnknownValue (!ce.PARA[8], 'celsius') returns 33.8 fahrenheit • PARA[8] = 1 fahrenheit !! comConvertUnknownValue (!ce.PARA[8], 'celsius') returns 1 fahrenheit If the conversion fails for any reason, the original input value is returned.		
Arguments	1	REAL	REAL dimensioned or undimensioned value to be returned in current units.
	2	STRING	Default unit to be applied to the input REAL if it is undimensioned.

Unit Conversions - Core Functions

Unit value conversions can be done using new UNITS and MEASURE objects.

Current units of a physical dimension (measure) can be determined from a MEASURE object:

Current units from a MEASURE object given the PTYPE	!dimension = object MEASURE ('dist') !currentunits = !dimension.currentunits()
Current units from a MEASURE object given an existing variable	! currentunits = !value.dimension().currentunits()

The units of a dimensioned value can be converted using the method `.convertUnits()`:

Convert from existing unit value to metres	<pre>!distvalue = 123mm !newvar = !distvalue.convertunits('metre') q var !newvar <REAL> 0.123metre</pre>
Using <code>.convertUnits()</code> to impose units on non-dimensioned values. Original value is assumed to be in the database units of the given dimension	<pre>!value = 123 !newvar = !value.convertunits('metre') q var !newvar <REAL> 0.123metre !value = 100 !newvar = !value.convertunits('degF') q var !newvar <REAL> 212degF</pre>
Using <code>.currentUnits()</code> to convert a value in the given units to the current working units. Current working units are mm and degC in these examples.	<pre>!value = 123 !newvar = !value.currentunits('metre') q var !newvar <REAL> 123000mm !value = 212 !newvar = !value.currentunits('degF') q var !newvar <REAL> 100degC</pre>

2.7 Units in Schematic Model Manager

2.7.1 Dimension Support in Schematic Model Manager Prior to 12.1

Schematic Model Manager allows the storage of data against the schematic elements as UDAs. A selection of UDAs is provided as part of the installation, some are mandatory and some are optional. UDAs can be used in Schematic Model Manager to store dimensioned data for example temperatures and pressures. Users can create their own UDAs, in Lexicon, for use with Schematic Model Manager and these can also hold dimensioned data.

Prior to 12.1, Schematic Model Manager implemented special units support for Angle, Area, Pressure, Temperature, Volume and Weight values that could be included in the ISO15926 format import file. Units UDAs were provided as mandatory UDAs in Schematic Model Manager and were attributes on each Diagram element (SCDIAG). The chosen units for these dimensioned quantities could be set in the Project Options form in Schematic Model Manager.

The project options file stored units that were currently used in Schematic Model Manager. These could be changed during the life of a project but doing so did not update the data that had already been loaded, data already in the database had to be re-imported for the new settings to be applied. This means that data could be stored in different units on different elements, for example pressure could be in Pascal for elements imported with one diagram but the units for pressure could then have been changed to be Bar and a second diagram then imported where the pressure was stored in Bar. The units that were used during the import of a diagram were stored in the Units UDAs on the diagram. Some elements types have a reference to the diagram(s) that they are on but for other element types this needs to be worked out by navigating the hierarchy.

Attribute Mappings are used during the import of diagrams to Schematic Model Manager to map data in the diagram files to attributes, including provided and custom UDAs, on schematic elements. Dimensioned attributes had mappings which include an Attribute Type which would correspond to a mandatory Unit UDA. During the import process the attribute mappings were used to determine which attributes were populated with data from the diagram file. If the attribute was dimensioned the unit for that attribute in the diagram file (which could be specific to the attribute value or set across the whole diagram) was used to convert the value to the correct value for the units specified in Schematic Model Manager.

2.7.2 Upgrade of Dimensioned Data for Schematic Model Manager in 12.1

In 12.1 the new units capabilities mean that the special units support in Schematic Model Manager is no longer required. Data imported in 12.1 will be stored in the appropriate units consistent with the data read from the ISO15926 import file.

Schematic data imported into Schematic Model Manager prior to 12.1 must be upgraded to use new Units functionality, but this process will be handled separately to the main upgrade process. A check is performed automatically on entry to Schematic Model Manager and the user will be warned if an upgrade is required. The upgrade process must be carefully considered by project administrators as it can affect multiple projects and locations. Firstly, schematic data is scanned to identify changes required. Secondly, UDA definitions are updated for the appropriate units. Thirdly, the changes identified are applied to the schematic data.

Refer to Schematic Model Manager User Guide for details of the upgrade process.

2.8 Units and UDAS

Careful consideration is needed for dimensioned data stored in UDAs prior to 12.1. It is likely that a conversion process should be run on such data in order to get correct results when accessing that data with the new units functionality in 12.1. The exact approach will depend on the available information and the assumptions in place when that data was stored. Here are two approaches that could be taken in different scenarios.

- UDA values may have been stored as real numbers with no units information, with the units being assumed by the consumer of that data. For example, a Design Pressure may have been stored as 10 and the assumption in the application using it was that this meant 10 bar. Once the project has been upgraded to 12.1, the project may take the decision to enhance their application to work with pressures rather than real numbers, and so the data would need to be upgraded. The approach would be to scan the database for all values of the particular UDA and write out a macro that went to each affected element and restated its value with a unit qualifier. Then the UDA definition could be updated to make it a pressure, and the macro run to upgrade all the values.
- UDA values may have been stored as value and unit pairs. In this scenario each stored quantity has two UDAs, one real number for the value and one text for the unit qualifier. The approach is then similar to that above, but the unit qualifier is read in each case from the text UDA value. Once the data has been upgraded, the UDA for the unit qualifier can be removed.

Other scenarios are likely to be broadly similar to these, and the principle is that the value is restated with the intended unit qualifier so that the appropriate conversion to database stored units is made. The important point is that any impact on applications using that data should be fully understood before any change is made.